Graphs and Graph Algorithms

A complete competitive programming course

Contents

How to Use This Material **6**
0.1 Who this is for . . . 6
0.2 What you will be able to do at the end . . . 6
0.3 The parts . . . 6
0.4 The graphs used throughout . . . 7
0.5 How each part is organised . . . 9
0.6 Conventions used . . . 9
0.7 Notes for teaching from this material . . . 9

1 Graph Fundamentals **11**
1.1 Why graphs exist . . . 11
1.2 Definition . . . 11
1.3 Terminology . . . 12
1.4 Directed and undirected . . . 12
1.5 Weighted and unweighted . . . 13
1.6 Graph versus tree . . . 13
1.7 Families of graphs worth naming . . . 14
1.8 Reading the constraints . . . 15
1.9 Common mistakes . . . 15
1.10 Check yourself . . . 15

2 Representing Graphs in C++ **16**
2.1 Adjacency list . . . 16
2.2 Adjacency matrix . . . 17
2.3 Edge list . . . 18
2.4 Comparison . . . 18
2.5 Reading input, the four standard shapes . . . 19
2.6 Grids: the graph you never build . . . 20
2.7 Storing visited and distance arrays . . . 20
2.8 Memory arithmetic, done once . . . 21
2.9 Common mistakes . . . 21
2.10 Check yourself . . . 21

3 Depth First Search **22**
3.1 What DFS does . . . 22
3.2 Why the visited array is not optional . . . 22
3.3 Recursive DFS . . . 23
3.4 Dry run on G1 . . . 23
3.5 Covering the whole graph . . . 24
3.6 Complexity . . . 24
3.7 Recursion depth, the practical danger . . . 25

3.8 Iterative DFS . . . 25
3.9 The DFS tree . . . 26
3.10 Entry and exit times . . . 26
3.11 What DFS is good for . . . 27
3.12 Common mistakes . . . 27
3.13 Codeforces problems that test exactly this . . . 28
3.14 Check yourself . . . 28

4 Breadth First Search **29**
4.1 What BFS does . . . 29
4.2 Why the layers are shortest distances . . . 29
4.3 The code . . . 29
4.4 Dry run on G1 . . . 30
4.5 Reconstructing the path . . . 31
4.6 BFS on a grid . . . 31
4.7 Multi-source BFS . . . 32
4.8 BFS from every component . . . 33
4.9 BFS or DFS . . . 33
4.10 Common mistakes . . . 33
4.11 Codeforces problems that test exactly this . . . 34
4.12 Check yourself . . . 34

5 Connected Components and Flood Fill **35**
5.1 The pattern . . . 35
5.2 Labelling every vertex with its component . . . 35
5.3 Flood fill on a grid . . . 36
5.4 Variations you should be able to write immediately . . . 38
5.5 Common mistakes . . . 38
5.6 Codeforces problems that test exactly this . . . 38
5.7 Check yourself . . . 38

6 Cycle Detection **40**
6.1 Undirected graphs . . . 40
6.2 Directed graphs . . . 42
6.3 Summary table . . . 44
6.4 Common mistakes . . . 44
6.5 Codeforces problems that test exactly this . . . 44
6.6 Check yourself . . . 45

7 Bipartite Graphs **46**
7.1 What bipartite means . . . 46
7.2 The theorem . . . 46
7.3 Two colouring by BFS . . . 47
7.4 Using the colouring afterwards . . . 48
7.5 Common mistakes . . . 49
7.6 Codeforces problems that test exactly this . . . 49
7.7 Check yourself . . . 49

8 Topological Sort and DP on a DAG **50**

8.1 The problem . . . 50
8.2 Kahn's algorithm, the BFS way . . . 50
8.3 The DFS way . . . 52
8.4 Lexicographically smallest topological order . . . 53
8.5 DP on a DAG . . . 53
8.6 Common mistakes . . . 54
8.7 Codeforces problems that test exactly this . . . 55
8.8 Check yourself . . . 55

9 Disjoint Set Union 56
9.1 The problem it solves . . . 56
9.2 The idea . . . 56
9.3 Version 1, the naive one . . . 56
9.4 Improvement 1: path compression . . . 57
9.5 Improvement 2: union by size . . . 57
9.6 Complexity . . . 58
9.7 Dry run: G1 edge by edge . . . 58
9.8 What DSU is used for . . . 59
9.9 Common mistakes . . . 59
9.10 Codeforces problems that test exactly this . . . 60
9.11 Check yourself . . . 60

10 Minimum Spanning Tree 61
10.1 The problem . . . 61
10.2 The working example, G3 . . . 61
10.3 The one idea behind both algorithms . . . 61
10.4 Kruskal's algorithm . . . 62
10.5 Prim's algorithm . . . 63
10.6 Kruskal or Prim . . . 65
10.7 Useful variants . . . 65
10.8 Common mistakes . . . 65
10.9 Codeforces problems that test exactly this . . . 66
10.10 Check yourself . . . 66

11 Shortest Paths 67
11.1 Choosing the algorithm . . . 67
11.2 1. Unweighted: BFS . . . 67
11.3 2. Weights 0 and 1: 0-1 BFS . . . 67
11.4 3. Non-negative weights: Dijkstra . . . 69
11.5 4. Negative weights: Bellman-Ford . . . 71
11.6 5. All pairs: Floyd-Warshall . . . 72
11.7 Comparison . . . 73
11.8 Common mistakes . . . 74
11.9 Codeforces problems that test exactly this . . . 74
11.10 Check yourself . . . 75

12 Trees as Graphs 76
12.1 What changes and what does not . . . 76
12.2 The tree used in this part . . . 76

12.3 Rooting a tree . . . 77
12.4 Subtree DP . . . 77
12.5 Tree diameter . . . 78
12.6 LCA and distances . . . 79
12.7 Euler tour: subtree becomes a range . . . 80
12.8 Common mistakes . . . 81
12.9 Codeforces problems that test exactly this . . . 81
12.10 Check yourself . . . 81

13 The DFS Tree, Bridges and Strong Connectivity 82
13.1 Recap: the DFS tree . . . 82
13.2 tin and low . . . 82
13.3 Bridges . . . 83
13.4 Articulation points . . . 84
13.5 Strongly connected components . . . 85
13.6 Common mistakes . . . 87
13.7 Codeforces problems that test exactly this . . . 87
13.8 Check yourself . . . 87

14 How to Think About Graph Problems 89
14.1 The ten questions . . . 89
14.2 The modelling catalogue . . . 90
14.3 The decision guide . . . 92
14.4 Pattern to problem map . . . 92
14.5 Three worked examples . . . 93
14.6 When you are stuck . . . 94
14.7 Habits of strong contestants . . . 94
14.8 Check yourself . . . 94

15 Practice Problem Sets 95
15.1 How to use this set . . . 95
15.2 A note on where to practise . . . 95
15.3 Level 1: first graph problems . . . 95
15.4 Level 2: easy . . . 96
15.5 Level 3: easy plus . . . 96
15.6 Level 4: medium . . . 97
15.7 Must Solve . . . 98
15.8 A six week plan . . . 99
15.9 Contest checklist . . . 99

16 Revision Sheet 100
16.1 Facts in one box . . . 100
16.2 Setup and input . . . 100
16.3 DFS . . . 101
16.4 BFS . . . 101
16.5 Components and flood fill . . . 102
16.6 Cycle detection . . . 102
16.7 Bipartite check . . . 102
16.8 Topological sort . . . 103

16.9 DSU 103
16.10 MST 104
16.11 Shortest paths 104
16.12 Trees 105
16.13 Bridges, articulation points, SCC 106
16.14 Complexity cheat sheet 107
16.15 Which approach should I think of first 107
16.16 The mistakes that cost the most 108
16.17 The four questions to ask on any new graph problem 108

How to Use This Material

0.1 Who this is for

Someone who has finished the Binary Trees course, or who is comfortable with recursion, `vector`, `queue`, `stack`, `map` and basic complexity analysis, and who has never studied graphs.

The target is competitive programming. Everything here is written the way you would actually write it in a contest: short, fast, 1-indexed, `scanf`-free but `cin`-with-fast-IO, global arrays where that is simpler. Interview style is mentioned where it differs, but contests are the priority.

0.2 What you will be able to do at the end

Read a Codeforces problem, notice that it is a graph problem even when the word "graph" never appears, build the graph, choose the right algorithm, and implement it correctly the first time. Concretely, by the end you can solve most graph problems rated up to about 1600, and you will understand the ideas behind harder ones.

The four questions this course trains you to ask:

```
What are the vertices, and what are the edges?
Is it directed? Is it weighted? Can it have cycles?
Do I need reachability, distance, ordering, or connectivity?
Which of the eight standard algorithms answers that question?
```

Modelling comes first. Half of all contest graph problems are easy once you see what the graph is, and impossible before that.

0.3 The parts

Part	Topic	What it gives you
1	Graph Fundamentals	vocabulary, graph versus tree, special graph families
2	Representing Graphs in C++	adjacency list, matrix, edge list, grids, fast input
3	Depth First Search	the visited array, recursion, DFS tree, entry and exit times
4	Breadth First Search	shortest paths in unweighted graphs, path reconstruction
5	Components and Flood Fill	counting, sizing and colouring components, grids

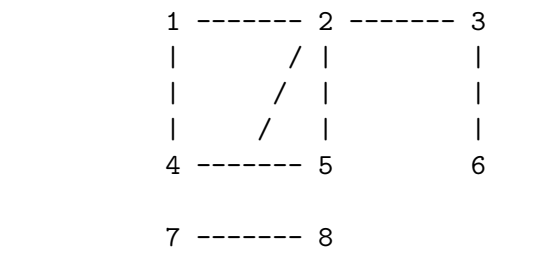
Part	Topic	What it gives you
6	Cycle Detection	undirected and directed, and how they differ
7	Bipartite Graphs	two colouring, odd cycles
8	Topological Sort	Kahn, DFS ordering, DP on a DAG
9	Disjoint Set Union	the CP workhorse, path compression, union by size
10	Minimum Spanning Tree	Kruskal and Prim
11	Shortest Paths	BFS, 0-1 BFS, Dijkstra, Bellman-Ford, Floyd-Warshall
12	Trees as Graphs	rooting, subtree DP, diameter, LCA by binary lifting
13	The DFS Tree	bridges, articulation points, strongly connected components
14	How to Think About Graph Problems	modelling, pattern recognition, decision guide
15	Practice Problem Sets	graded Codeforces problems with hints
16	Revision Sheet	every template on a few pages

Read them in order. Parts 3, 4 and 9 are the spine. If you are short of time, never skip those.

0.4 The graphs used throughout

Just like the trees course used one tree everywhere, this course uses four small graphs everywhere. Learn them now.

0.4.1 G1, the standard undirected graph



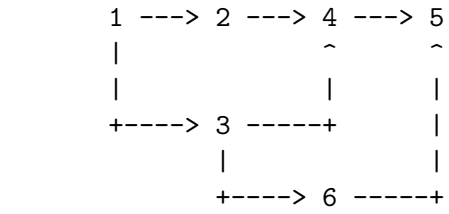

```
n = 8 vertices, m = 8 edges
edges: 1-2  1-4  2-3  2-4  2-5  3-6  4-5  7-8

adjacency (sorted):
1: 2 4
2: 1 3 4 5
3: 2 6
4: 1 2 5
```

```
5: 2 4
6: 3
7: 8
8: 7

DFS order from 1 : 1 2 3 6 4 5
BFS order from 1 : 1 2 4 3 5 6
BFS distances    : d[1]=0 d[2]=1 d[4]=1 d[3]=2 d[5]=2 d[6]=3
components       : {1,2,3,4,5,6} and {7,8}
contains cycles  : 1-2-4-1 and 2-4-5-2
bipartite        : no, there are odd cycles (triangles)
```

0.4.2 G2, the standard DAG




```
edges: 1->2  1->3  2->4  3->4  3->6  4->5  6->5
indegrees: 1:0  2:1  3:1  4:2  5:2  6:1
topological order (Kahn, queue by smallest first): 1 2 3 4 6 5
```

0.4.3 G3, the standard weighted graph

This one has crossing edges, so instead of a cramped drawing, here is the layout plus the edge list. Draw it once on paper and keep that page.

```
place the vertices like this and join them:

        (1)                 (2)
             (3)
                                 (4)
        (6)                 (5)

edges (undirected, weighted):
1-2 : 7      1-3 : 9      1-6 : 14
2-3 : 10     2-4 : 15
3-4 : 11     3-6 : 2
4-5 : 6      5-6 : 9

Dijkstra from vertex 1:
d[1]=0  d[2]=7  d[3]=9  d[6]=11  d[4]=20  d[5]=20

Minimum spanning tree (Kruskal):
3-6 (2), 4-5 (6), 1-2 (7), 1-3 (9), 5-6 (9)      total weight 33
```

0.4.4 G4, the standard grid

```
. . # . .
. # # . .
. . . . .
# # . # .
```

Four rows, five columns, `#` is a wall, `.` is free. Row and column indices are 0-based. Grid problems are graph problems where the graph is never written down, and Part 2 explains why.

0.5 How each part is organised

1. what the thing means, in plain words
2. a picture, on one of the four standard graphs
3. the algorithm in pseudocode
4. C++ implementation in contest style
5. a dry run
6. time and space complexity, with the reason
7. common mistakes
8. Codeforces problems that test exactly this
9. Check Yourself questions

The Check Yourself questions are the actual test of whether you understood. Answer them before moving on.

0.6 Conventions used

- **n** is the number of vertices, **m** is the number of edges.
- Vertices are numbered **1 to n**, because that is what Codeforces input gives you. Grids are 0-indexed because that is what arrays give you.
- Adjacency lists are `vector<int> adj[N]` or `vector<vector<int>> adj(n+1)`.
- `INF` is `1e9` for `int` and `1e18` for `long long`.
- Code assumes these three lines at the top:

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 200005;
```

- Complexity is written in terms of n and m. **O(n + m)** is the single most common answer in this course.

0.7 Notes for teaching from this material

- Parts 1 and 2 are one session together. Do not spend a whole class on definitions.
- Part 3 and Part 4 are one session each, and they must be taught with live coding.
- Part 9 (DSU) always takes longer than expected. Budget a full session.

- Part 11 is two sessions: unweighted plus 0-1 BFS, then Dijkstra plus the rest.
- Part 13 can be skipped for a beginner batch. It is marked as advanced.
- Part 14 lands best after students have failed at two or three modelling problems.

Part 1

Graph Fundamentals

Vocabulary, and how a graph differs from a tree

1.1 Why graphs exist

A tree can express hierarchy: one parent, many children, no way back up except the way you came. That is a strong restriction, and most real relationships break it.

- Cities joined by roads. A city has no parent. You can drive in a circle.
- Friendships. If A is a friend of B then B is a friend of A. Not a hierarchy.
- Course prerequisites. Many courses depend on many others, and the dependency has a direction.
- Web pages linking to each other. Circular links are normal.
- States of a puzzle, where a move takes you from one state to another.

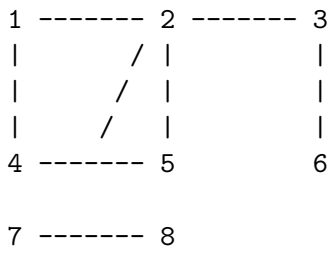
A graph is the most general "things and connections between them" structure. Trees, linked lists, grids and hierarchies are all special cases of it. That generality is exactly why graphs are the single largest topic in competitive programming.

1.2 Definition

A **graph** is a pair `G = (V, E)`:

- `V` is a set of **vertices**, also called nodes. We number them 1 to n.
- `E` is a set of **edges**, each edge joining two vertices.

That is the whole definition. There is no root, no parent, no order among the neighbours of a vertex, and no guarantee that everything is connected.



This is G1: n = 8, m = 8. Notice three things a tree would never allow: a cycle (1-2-4-1), a vertex with four neighbours and no notion of "children", and a second piece (7-8) not connected to the rest.

1.3 Terminology

Everything below refers to G1.

Vertex (plural vertices): a single item. Also called a node.

Edge: a connection. We write it `1-2` for undirected and `1->2` for directed.

Adjacent: two vertices joined by an edge. 1 and 2 are adjacent. 1 and 3 are not.

Neighbour: same thing from one vertex's point of view. The neighbours of 2 are 1, 3, 4, 5.

Degree of a vertex: how many edges touch it. `deg(2) = 4`, `deg(6) = 1`, `deg(7) = 1`.

Handshake lemma: the sum of all degrees equals `2m`, because every edge adds one to the degree of each of its two endpoints. For G1: 2 + 4 + 2 + 3 + 2 + 1 + 1 + 1 = 16 = 2 * 8. Useful for checking that you read the input correctly, and for the fact that the total work of visiting all adjacency lists is O(m), not O(nm).

Walk: any sequence of vertices where consecutive ones are joined by an edge. Repeats allowed.

Path: a walk with no repeated vertex. `1 - 2 - 3 - 6` is a path.

Cycle: a path that returns to its starting vertex, using at least 3 distinct vertices in an undirected graph. `1 - 2 - 4 - 1` is a cycle.

Connected: two vertices are connected if some path joins them. 1 and 6 are connected. 1 and 7 are not.

Connected component: a maximal group of mutually connected vertices. G1 has two components: `{1,2,3,4,5,6}` and `{7,8}`.

Simple graph: no self loops (an edge from a vertex to itself) and no multiple edges between the same pair. Most problems give simple graphs, but read the statement, because "there may be multiple roads between two cities" is a real sentence that appears in real problems.

Subgraph: some of the vertices plus some of the edges among them.

Sparse graph: m is close to n. **Dense** graph: m is close to n^2. This decides your representation, as Part 2 explains.

1.4 Directed and undirected

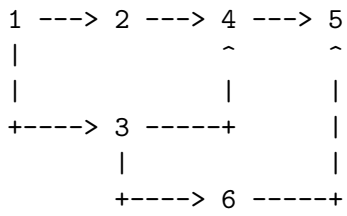
An **undirected** edge can be walked both ways. A **directed** edge (also called an arc) can be walked only in its direction.

```
undirected            directed
   1 --- 2              1 --> 2
```

In a directed graph, degree splits in two:

- **outdegree(v)**: number of edges leaving v
- **indegree(v)**: number of edges entering v

For G2:




```
outdegree: 1:2  2:1  3:2  4:1  6:1  5:0
indegree : 1:0  2:1  3:1  4:2  6:1  5:2
```

The single most common bug in beginner graph code is storing an undirected edge only once. **An undirected edge must be pushed into both adjacency lists.** Say it out loud every time you write the input loop.

Directed graphs also complicate connectivity. In a directed graph, "u can reach v" does not imply "v can reach u", which is why we need the notion of strongly connected components in Part 13.

1.5 Weighted and unweighted

A **weight** is a number attached to an edge: length, cost, time, capacity. Unweighted means every edge counts as 1.

This distinction decides your shortest path algorithm, and nothing else in the course:

```
unweighted                -> BFS,              O(n + m)
weights only 0 or 1       -> 0-1 BFS (deque), O(n + m)
non-negative weights      -> Dijkstra,         O(m log n)
any weights, may be < 0   -> Bellman-Ford,     O(n m)
all pairs, small n        -> Floyd-Warshall,   O(n^3)
```

Learn that block now. In Part 11 you will only be filling in the details.

1.6 Graph versus tree

You have already studied binary trees. Almost everything transfers, with four changes that cause almost all beginner bugs.

	Tree (from the previous course)	General graph
Structure	`TreeNode*` with `left`, `right`	adjacency list `vector<int> adj[N]`
Root	there is one	there is none, you pick a start vertex
Parent	each node has one	a vertex can have many predecessors
Cycles	impossible	possible, this is the big one

	Tree (from the previous course)	General graph
Revisiting a node	cannot happen	happens constantly, so you need `visited[]`
Connectivity	the whole tree is one piece	may be several components, so loop over all start vertices
Number of edges	exactly n - 1	anywhere from 0 to n(n-1)/2
Path between two nodes	unique	many, possibly none
Recursion depth	O(height)	O(n), a path graph is a chain of n calls
Traversal cost	O(n)	O(n + m)

The four practical consequences, and you should write them on the first page of your notebook:

1. **Always keep a `visited` array.** Without it, DFS on a cyclic graph never terminates.
2. **Always loop over all vertices** when you want to cover the whole graph, because one DFS only covers one component.
3. **Complexity is O(n + m), not O(n).** You touch every vertex once and every adjacency entry once, and every undirected edge appears in two lists.
4. **Recursion can go n deep.** With n = 2 * 10^5 and heavy stack frames, plain DFS can crash. Keep the frame small, or write it iteratively.

A tree is not a different data structure from a graph. **A tree is a connected undirected graph with no cycles**, equivalently a connected graph with exactly n - 1 edges. Part 12 revisits the whole trees course in graph language, and everything you learned there still works.

1.7 Families of graphs worth naming

Tree: connected, acyclic, n - 1 edges, unique path between any two vertices. Adding any edge creates exactly one cycle, and removing any edge disconnects it.

Forest: a disjoint set of trees. If a graph has c components, no cycles and n vertices, it has exactly n - c edges.

DAG, directed acyclic graph: directed, no directed cycle. Every DAG has at least one vertex of indegree 0. This is the family where topological sorting and DP on the graph live (Part 8).

Bipartite graph: the vertices split into two sides so that every edge goes between the sides, never inside one side. Equivalently, no odd cycle (Part 7).

Complete graph `K_n`: every pair joined. It has `n(n-1)/2` edges, which is the maximum for a simple undirected graph. A directed simple graph maxes out at `n(n-1)`.

Grid graph: an `r` by `c` board where each cell is a vertex joined to its 4 (or 8) neighbours. Almost never given as an edge list, so Part 2 shows the standard trick.

Functional graph: every vertex has exactly one outgoing edge, for example `a[i]` meaning “i points to a[i]”. Such a graph is a set of cycles, each with trees hanging off it. Permutations are the special case where it is exactly a set of cycles.

Implicit or state graph: the vertices are states of some process (a configuration of a puzzle, a position plus a remaining fuel amount) and the edges are legal moves. You never build the graph, you generate neighbours on demand. Part 14 is largely about spotting these.

1.8 Reading the constraints

Contest constraints tell you the intended algorithm before you have read the statement properly.

```
n <= 20                 bitmask over subsets, 2^n states
n <= 500                O(n^3) is fine: Floyd-Warshall, or an adjacency matrix
n <= 2000               O(n^2) or O(n m) on a sparse graph
n, m <= 2 * 10^5        O(n + m) or O(m log n). Adjacency list, nothing heavier
n <= 10^6               O(n + m) with fast IO, keep constants small
weights up to 10^9      sums need long long
```

If n and m are both up to 2 * 10^5, an adjacency matrix would need 4 * 10^{10} cells. That is not a close call, it is impossible. Sparse means list.

1.9 Common mistakes

- **Adding an undirected edge only once.** Half your edges silently vanish.
- **Forgetting `visited`.** Infinite recursion, or exponential blowup on graphs with many cycles.
- **Running one DFS from vertex 1 and assuming the graph is covered.** Only true if the graph is connected, and the statement must say so.
- **Sizing arrays with n instead of n + 1** when vertices are numbered 1 to n.
- **Using an adjacency matrix when n is 10^5.**
- **Assuming there is a path.** In a disconnected graph the answer is often "-1", and problems test exactly that case.
- **Assuming the graph is simple.** Self loops and multiple edges break naive cycle detection.
- **Assuming a connected graph with n vertices and n - 1 edges is a tree without checking connectivity.** You need both facts, though either one plus acyclicity implies the other.
- **Ignoring that m can be 0.**

1.10 Check yourself

1. G1 has 8 edges. What is the sum of all degrees, and why can you answer without adding them up one by one?
2. A graph has n = 10 vertices and 3 components and no cycles. How many edges does it have?
3. Why does a general graph need a `visited` array when a binary tree traversal does not?
4. What is the maximum number of edges in a simple undirected graph on n vertices? On a directed one?
5. The constraints say n <= 400 and there are queries asking for the distance between any two vertices. Which algorithm is being hinted at?
6. Give a real-world relationship that is directed and one that is undirected.
7. Why is the traversal complexity O(n + m) and not O(n * m)?

Part 2

Representing Graphs in C++

Adjacency lists, matrices, edge lists, grids

In the trees course a node was a struct with two pointers, and the structure was the memory layout. Graphs are different. There is no `GraphNode`. A graph is stored as a table of who is next to whom, and you get to choose the table.

There are three standard choices. In competitive programming you will use the first one ninety percent of the time.

2.1 Adjacency list

For every vertex, store the list of its neighbours.

```
G1:
1: 2 4
2: 1 3 4 5
3: 2 6
4: 1 2 5
5: 2 4
6: 3
7: 8
8: 7
```

```cpp
const int N = 200005;
vector<int> adj[N];
int n, m;

int main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);      // undirected: BOTH directions
    }
}
```

For a directed graph, delete the second push_back. That single line is the whole difference, and forgetting it is the most common bug in this entire course.

Iterating over the neighbours of `u`:

```
for (int v : adj[u]) {
    // v is a neighbour of u
}
```

Memory: O(n + m) for a directed graph, O(n + 2m) for an undirected one, because each undirected edge is stored twice. **Iterate neighbours of u**: O(deg(u)), which is optimal. **Check whether edge (u,v) exists**: O(deg(u)), which is slow. If a problem asks that repeatedly, use a `set` or a matrix.

2.1.1 The vector version, for multiple test cases

Codeforces problems very often have `t` test cases in one input. Global arrays then have to be cleared, and clearing all N entries per test is O(N) per test, which times out. Two safe options:

```
// Option A: local vector, sized per test. Cleanest and safe.
int n, m;
cin >> n >> m;
vector<vector<int>> adj(n + 1);

// Option B: global array, but clear only what you used
for (int i = 1; i <= n; i++) adj[i].clear();
```

Option A is what you should default to. It costs a little allocation time and saves you from the classic "passes sample, fails test 2" bug where leftover edges from the previous test case are still in the lists.

2.1.2 Weighted graphs

```
vector<pair<int,int>> adj[N];      // (neighbour, weight)

adj[u].push_back({v, w});
adj[v].push_back({u, w});

for (auto [v, w] : adj[u]) { ... }              // C++17
for (auto p : adj[u]) { int v = p.first, w = p.second; ... }   // any version
```

Keep the pair as `(neighbour, weight)`, not `(weight, neighbour)`. Dijkstra's priority queue wants the opposite order, and mixing the two conventions in one file is a reliable way to lose forty minutes.

2.2 Adjacency matrix

A 2D array where `g[u][v]` is 1 if the edge exists.

```
G1 as a matrix (rows and columns 1..8):

    1 2 3 4 5 6 7 8
  1 0 1 0 1 0 0 0 0
  2 1 0 1 1 1 0 0 0
  3 0 1 0 0 0 1 0 0
  4 1 1 0 0 1 0 0 0
  5 0 1 0 1 0 0 0 0
  6 0 0 1 0 0 0 0 0
  7 0 0 0 0 0 0 0 1
```

```
  8   0 0 0 0 0 0 1 0
```

```
const int MAXN = 505;
int g[MAXN][MAXN];

g[u][v] = 1;
g[v][u] = 1;       // undirected
```

Notice the matrix of an undirected graph is symmetric, and its diagonal is 0 for a simple graph.

Memory: $O(n^2)$. With n = 500 that is 250000 ints, about 1 MB. Fine. With $n = 10^5$ it is 10^{10} ints. Impossible. **Check whether edge (u,v) exists**: O(1). This is its only real advantage. **Iterate neighbours of u**: O(n), even if u has one neighbour. On a sparse graph this makes every algorithm $O(n^2)$ instead of O(n + m).

Use a matrix when: n <= about 500, or the problem is naturally dense, or you are running Floyd-Warshall (Part 11), which needs a matrix anyway.

For weighted matrices, store the weight and use a large sentinel for "no edge":

```
const int INF = 1e9;
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++)
        g[i][j] = (i == j ? 0 : INF);
```

2.3 Edge list

Just store the edges.

```
struct Edge { int u, v, w; };
vector<Edge> edges;
edges.push_back({u, v, w});
```

Useless for traversal, because finding the neighbours of a vertex means scanning everything. Perfect for two specific algorithms:

- **Kruskal's MST** (Part 10), which sorts all edges by weight.
- **Bellman-Ford** (Part 11), which relaxes every edge n - 1 times.

Also handy when a problem asks you to answer something about edges themselves, such as "delete edge i and report the number of components".

2.4 Comparison

	Adjacency list	Adjacency matrix	Edge list
Memory	O(n + m)	$O(n^2)$	O(m)
Is (u,v) an edge?	O(deg u)	O(1)	O(m)
Loop over neighbours of u	O(deg u)	O(n)	O(m)
Add an edge	O(1)	O(1)	O(1)

	Adjacency list	Adjacency matrix	Edge list
Good for	almost everything	dense, small n, Floyd	Kruskal, Bellman-Ford

Default to the adjacency list. Switch only for a stated reason.

2.5 Reading input, the four standard shapes

```
// 1. undirected, unweighted
cin >> n >> m;
vector<vector<int>> adj(n + 1);
for (int i = 0; i < m; i++) {
    int u, v; cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}

// 2. directed, unweighted
for (int i = 0; i < m; i++) {
    int u, v; cin >> u >> v;
    adj[u].push_back(v);
}

// 3. undirected, weighted
vector<vector<pair<int,int>>> adj(n + 1);
for (int i = 0; i < m; i++) {
    int u, v, w; cin >> u >> v >> w;
    adj[u].push_back({v, w});
    adj[v].push_back({u, w});
}

// 4. a tree given as parent array (very common)
// p[i] is the parent of vertex i, for i = 2..n
vector<vector<int>> adj(n + 1);
for (int i = 2; i <= n; i++) {
    int p; cin >> p;
    adj[p].push_back(i);
    adj[i].push_back(p);
}
```

Shape 4 deserves attention. Many Codeforces problems hand you a tree as `n - 1` parent values rather than edges. Build it as an ordinary undirected graph anyway, then root it at 1 during the DFS. Part 12 explains why that is better than trying to use the parent array directly.

2.5.1 Fast input

With m up to 2 * 10^5 or more, `cin` without this line is genuinely slow enough to fail:

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

Put those two lines at the top of `main` in every solution. They cost nothing and save you from a class

of time limit failures that have nothing to do with your algorithm.

2.6 Grids: the graph you never build

A grid problem gives you something like

```
4 5
..#..
.##..
.....
##.#.
```

Every free cell is a vertex, and each cell is joined to the free cells directly above, below, left and right. There are up to 4 * r * c edges, and writing them into an adjacency list is a waste of memory and typing. Instead, generate neighbours on demand:

```
int R, C;
vector<string> g;

int dx[4] = {-1, 1, 0, 0};
int dy[4] = { 0, 0,-1, 1};

// neighbours of cell (x, y):
for (int k = 0; k < 4; k++) {
    int nx = x + dx[k], ny = y + dy[k];
    if (nx < 0 || nx >= R || ny < 0 || ny >= C) continue;   // off the board
    if (g[nx][ny] == '#') continue;                          // wall
    // (nx, ny) is a neighbour
}
```

Everything in this course works on grids with that loop substituted for `for (int v : adj[u])`. BFS on a grid is BFS. Components on a grid are components. Nothing new is happening.

For 8 directions:

```
int dx[8] = {-1,-1,-1, 0, 0, 1, 1, 1};
int dy[8] = {-1, 0, 1,-1, 1,-1, 0, 1};
```

If you ever need a single integer id for a cell, use `id = x * C + y`, and back again with `x = id / C`, `y = id % C`. That is needed for DSU on a grid, and occasionally for a `visited` set.

Order matters for readability, not correctness. Any order of the four directions gives the same BFS distances.

2.7 Storing visited and distance arrays

```
vector<int> vis(n + 1, 0);
vector<int> dist(n + 1, -1);          // -1 means "not reached"
vector<int> par(n + 1, 0);            // for path reconstruction
vector<long long> d(n + 1, 1e18);     // for Dijkstra
```

Two notes that will save you real debugging time:

- Size everything `n + 1` when vertices are 1-indexed. Sizing it `n` and then touching `vis[n]` is undefined behaviour that usually looks like a wrong answer, not a crash.
- Prefer `dist` initialised to `-1` over a separate `visited` array for BFS. One array, one meaning: "-1 means not visited". Fewer things to forget.

2.8 Memory arithmetic, done once

```
int is 4 bytes, long long is 8 bytes
vector<int> adj[200005] with 2m = 4*10^5 total entries
    ~ 1.6 MB of data plus vector overhead, comfortable

int g[500][500]     = 1 MB          fine
int g[5000][5000]   = 100 MB        usually too much
int g[100000][100000]                impossible

Typical Codeforces memory limit: 256 MB
```

2.9 Common mistakes

- **Pushing an undirected edge once.** Say it with me: both directions.
- **Not clearing global adjacency lists between test cases**, or clearing all N of them and timing out.
- **Sizing arrays `n` instead of `n + 1`.**
- **Using an adjacency matrix on a sparse graph with large n.**
- **Storing `(weight, neighbour)` in one place and `(neighbour, weight)` in another.**
- **Forgetting fast IO** and losing to the input reader rather than the algorithm.
- **Rebuilding a grid into an explicit adjacency list.** Works, wastes memory and time, and adds bugs.
- **Forgetting the bounds check before reading `g[nx][ny]`.** In C++ that reads out of bounds instead of throwing, so the failure is silent.

2.10 Check yourself

1. Why is an adjacency list preferred over a matrix when n = 10^5 and m = 2 * 10^5?
2. What is the memory cost of the adjacency list of an undirected graph with m edges, and why is it not O(m)?
3. Which two algorithms actually prefer an edge list?
4. Write the input loop for a directed weighted graph from memory.
5. On a grid, what does `id = x * C + y` accomplish, and when do you need it?
6. A problem has `t` test cases with total n up to 2 * 10^5. What is wrong with clearing a global `adj[200005]` in full at the start of each test?
7. Which representation makes "is there an edge between u and v" O(1), and what do you pay for it?

Part 3

Depth First Search

The visited array, the DFS tree, entry and exit times

3.1 What DFS does

Same idea as in the trees course: go as deep as you can, and back up only when stuck. In a maze, keep your hand on the left wall and walk.

The code even looks the same. Compare:

```
// tree version, from the previous course
void dfs(TreeNode* node) {
    if (node == NULL) return;
    dfs(node->left);
    dfs(node->right);
}

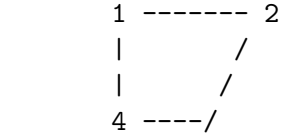
// graph version
void dfs(int u) {
    vis[u] = 1;
    for (int v : adj[u])
        if (!vis[v]) dfs(v);
}
```

Two differences, and they are the whole story:

1. A node had exactly two children. A vertex has any number of neighbours, so the two calls become a loop.
2. A tree had no cycles, so you could never arrive twice at the same node. A graph can, so we need `vis[]`.

3.2 Why the visited array is not optional

Run the graph code on G1 without `vis`, starting at 1:



`dfs(1)` calls `dfs(2)`, which calls `dfs(1)` (because 1 is a neighbour of 2), which calls `dfs(2)` again. The program never stops. On a graph with cycles, no visited array means infinite recursion.

Even on an acyclic directed graph it is not safe to drop it. A DAG can force exponential work:

```
1 -> 2 -> 4 -> 6 ...
 \    \    \
  \--> 3 --> 5 ...
```

Without marking, the number of paths explored can double at every layer. `vis[]` turns "explore every path" into "explore every vertex once", which is the difference between exponential and linear.

3.3 Recursive DFS

```cpp
const int N = 200005;
vector<int> adj[N];
bool vis[N];

void dfs(int u) {
    vis[u] = true;
    // work done on entering u (preorder position)

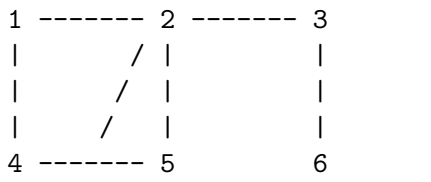
    for (int v : adj[u]) {
        if (!vis[v]) {
            dfs(v);
        }
    }

    // work done on leaving u (postorder position)
}
```

The preorder and postorder positions are exactly the ones from the trees course, and they are used for the same reasons: information flowing down goes in the first position, information coming back up goes in the second.

Mark `vis[u] = true` on **entry**, before the loop, not after it. Marking after the loop means a cycle re-enters `u` before it was marked, and you are back to infinite recursion.

3.4 Dry run on G1




```
adjacency, in sorted order:
1: 2 4        2: 1 3 4 5       3: 2 6
4: 1 2 5      5: 2 4           6: 3

dfs(1)  vis={1}                 look at 2 -> not visited
  dfs(2)  vis={1,2}             look at 1 -> visited, skip
                                look at 3 -> not visited
```

```
    dfs(3)  vis={1,2,3}         look at 2 -> visited, skip
                                look at 6 -> not visited
      dfs(6)  vis={1,2,3,6}     look at 3 -> visited, skip
      return from 6
    return from 3
                                back in 2, look at 4 -> not visited
    dfs(4)  vis={1,2,3,6,4}     look at 1 -> visited
                                look at 2 -> visited
                                look at 5 -> not visited
      dfs(5)  vis={1,2,3,6,4,5} look at 2 -> visited
                                look at 4 -> visited
      return from 5
    return from 4
                                back in 2, look at 5 -> visited
  return from 2
                                back in 1, look at 4 -> visited
return from 1

visit order: 1 2 3 6 4 5
```

Vertices 7 and 8 were never reached. One DFS explores one component, and nothing more. That is the second big difference from trees.

3.4.1 The order depends on the adjacency order

If the input had listed edge `1-4` before `1-2`, the order would have been `1 4 2 3 6 5`. Both are valid depth first orders. Never write a solution whose correctness depends on which neighbour comes first, unless you sorted the lists on purpose (which some problems, such as lexicographically smallest orderings, do require).

3.5 Covering the whole graph

```
for (int i = 1; i <= n; i++)
    if (!vis[i])
        dfs(i);
```

Four lines that appear in half of all graph solutions. Wrapping a counter around them counts components, which is Part 5.

3.6 Complexity

Time O(n + m).

The reason is worth stating properly, because students often guess O(n * m). Each vertex is entered at most once, because of `vis`. When a vertex `u` is entered, the loop runs `deg(u)` times. So the total loop work is the sum over all vertices of `deg(u)`, which by the handshake lemma is `2m` for an undirected graph and `m` for a directed one. Add the `n` entries and you get O(n + m).

Space O(n) for `vis` plus O(n) for the recursion stack in the worst case, on top of the O(n + m) the graph itself occupies.

3.7 Recursion depth, the practical danger

In a binary tree, recursion went as deep as the height, and a balanced tree gave you 20 frames. In a graph, a simple path

```
1 - 2 - 3 - 4 - ... - 200000
```

gives you 200000 nested calls. Each frame holds the parameters, saved registers and the iterator over the adjacency list. Typical stack limit on Codeforces is 256 MB, and plain DFS usually survives, but code with large local arrays inside the recursive function will not.

Rules that keep you safe:

- Never declare a big local array inside a recursive function. Use globals.
- Pass few parameters, and pass them by value if they are ints, or by reference if they are containers.
- If the problem has n up to 10^6, or you are on a judge with a small stack, write the iterative version.

3.8 Iterative DFS

```
void dfsIterative(int s) {
    stack<int> st;
    st.push(s);

    while (!st.empty()) {
        int u = st.top();
        st.pop();

        if (vis[u]) continue;      // may have been pushed several times
        vis[u] = true;

        for (int v : adj[u])
            if (!vis[v])
                st.push(v);
    }
}
```

Two subtleties:

1. **A vertex can be pushed more than once**, by different neighbours, before it is popped. So the check `if (vis[u]) continue;` after popping is required. Marking at push time also works and pushes fewer duplicates, but then the order of visits changes.
2. **The visit order is not identical to the recursive version.** The stack reverses the neighbour order, so this code visits the last neighbour first. To match exactly, iterate `adj[u]` in reverse when pushing. For most problems the order does not matter, but for a printed answer it does.

This simple version cannot do postorder work, because when you pop `u` you do not yet know when its subtree finishes. If you need the postorder position (for topological sort or SCC), either use recursion,

or push pairs of `(vertex, edge index)` and simulate the call stack properly. Recursion is almost always the better choice.

3.9 The DFS tree

Every DFS implicitly builds a tree: whenever `dfs(u)` calls `dfs(v)`, draw a tree edge `u - v`. For G1 starting at 1:

```
DFS tree                 non-tree edges
        1                1 - 4   (back edge)
        |                2 - 5   (back edge)
        2
       / \
      3   4
      |   |
      6   5
```

The component had 6 vertices and 7 edges. Five of them are tree edges (a tree on 6 vertices) and two are left over.

In an **undirected** graph, every non-tree edge joins a vertex to one of its own ancestors. Such an edge is called a **back edge**. There are no "cross edges" going sideways between different branches, and this is not an accident: if such an edge existed, DFS would have walked along it and made it a tree edge. That single fact is the foundation of cycle detection, bridges and articulation points.

In a **directed** graph, four kinds of non-tree edge are possible: back (to an ancestor), forward (to a descendant), and cross (to an already-finished vertex elsewhere). Only back edges indicate a cycle. Part 6 and Part 13 use this.

3.10 Entry and exit times

Give the DFS a clock. Record the time at which each vertex is entered and left.

```
int timer = 0;
int tin[N], tout[N];

void dfs(int u) {
    vis[u] = true;
    tin[u] = timer++;

    for (int v : adj[u])
        if (!vis[v]) dfs(v);

    tout[u] = timer++;
}
```

On G1 from vertex 1:

```
vertex : 1   2   3   6   4   5
tin    : 0   1   2   3   6   7
```

```
tout   : 11  10  5   4   9   8
```

The pairs nest like brackets:

```
1 [ 2 [ 3 [ 6 [ ] ] 4 [ 5 [ ] ] ] ]
```

This gives you a genuinely useful test, in O(1):

> `u` is an ancestor of `v` in the DFS tree if and only if `tin[u] <= tin[v]` and `tout[v] <= tout[u]`.

That single line is used for:

- checking ancestry without walking up the tree
- turning a subtree into a contiguous range of an array, so that subtree queries become range queries (the Euler tour trick, mentioned again in Part 12)
- bridges and articulation points, in Part 13

It is worth spending five minutes drawing the bracket picture until it feels obvious.

3.11 What DFS is good for

connected components	Part 5
cycle detection	Part 6
bipartite checking (2-colouring)	Part 7
topological sort	Part 8
subtree sizes and tree DP	Part 12
bridges, articulation points, SCC	Part 13
flood fill on grids	Part 5
counting paths in a DAG	Part 8

What DFS is **not** good for: shortest paths in an unweighted graph. DFS finds *a* path, almost never the shortest one. That is BFS, and it is the next part.

3.12 Common mistakes

- **Marking `vis[u]` after the loop instead of before it.** Infinite recursion.
- **Only running DFS from vertex 1.** Misses every other component.
- **Not resetting `vis` between test cases**, or resetting all 200005 entries every test and timing out.
- **Believing DFS gives shortest paths.** It does not.
- **Declaring `int cnt[100000]` inside the recursive function.** Stack death.
- **Assuming the DFS order is unique.** It depends on the input order of edges.
- **In the iterative version, forgetting the `if (vis[u]) continue;` after popping.**
- **Using `int` for a `timer` in a problem with 10^6 vertices and then also using `2 * timer`.** Fine for ints, but check.

3.13 Codeforces problems that test exactly this

- 977E, Cyclic Components. https://codeforces.com/problemset/problem/977/E
- 580C, Kefa and Park. DFS on a tree with a running counter. https://codeforces.com/problemset/problem/580/C
- 377A, Maze. Grid DFS with a counting twist. https://codeforces.com/problemset/problem/377/A

3.14 Check yourself

1. Why does the graph version of DFS need `vis` when the tree version did not?
2. What exactly goes wrong if you set `vis[u] = true` after the neighbour loop?
3. Give the DFS order of G1 starting from vertex 2, with sorted adjacency lists.
4. Why is DFS O(n + m) rather than O(n * m)?
5. In an undirected graph, why can a non-tree DFS edge only go to an ancestor?
6. Using `tin` and `tout`, how do you check in O(1) whether u is an ancestor of v?
7. Your DFS solution passes the samples and gets a runtime error on test 12 with n = 200000. Name two likely causes.

Part 4

Breadth First Search

Shortest paths in unweighted graphs

4.1 What BFS does

Same as the level order traversal from the trees course: finish everything at distance 1, then everything at distance 2, and so on. The queue is the same queue.

The difference from trees is again the `visited` check, and the payoff is bigger here, because BFS on a graph answers a question DFS cannot:

> **In an unweighted graph, BFS from `s` gives the minimum number of edges from `s` to every other vertex.**

That sentence is the reason BFS matters. Memorise it with its condition: **unweighted**. The moment edges have different weights, BFS is wrong and you need Part 11.

4.2 Why the layers are shortest distances

BFS processes vertices in non-decreasing order of distance. The argument is short enough to keep in your head:

- Layer 0 is `{s}`, at distance 0, obviously correct.
- Suppose every vertex at distance `k` has been placed in the queue and nothing further has been. When we pop those vertices and push their unvisited neighbours, any such neighbour must be at distance exactly `k + 1`: it cannot be closer, or it would already have been found in an earlier layer, and it is reachable in `k + 1` steps by construction.

So the first time BFS reaches a vertex, it reaches it optimally. That is why we set `dist` and mark visited **at push time**, not at pop time.

4.3 The code

```
vector<int> bfs(int s, int n, vector<vector<int>>& adj) {
    vector<int> dist(n + 1, -1);       // -1 means unreached, and doubles as "not visited"
    queue<int> q;

    dist[s] = 0;
    q.push(s);
```

```
    while (!q.empty()) {
        int u = q.front();
        q.pop();

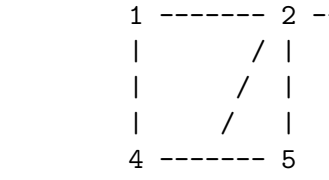
        for (int v : adj[u]) {
            if (dist[v] == -1) {       // first time we see v
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist;
}
```

Eleven lines, and you will write them a hundred times. Learn them until they come out without thinking.

Two design choices worth defending:

- **One array instead of two.** `dist[v] == -1` already means "not visited", so a separate `vis` array is redundant and gives you one more thing to forget.
- **Assign `dist[v]` at push time.** If you instead assign it when popping, the same vertex can be pushed by several neighbours before it is popped, and you get duplicates in the queue, wasted work, and in some variants a wrong answer.

4.4 Dry run on G1




```
1: 2 4     2: 1 3 4 5     3: 2 6     4: 1 2 5     5: 2 4     6: 3

push 1, dist[1]=0                    q = [1]

pop 1   neighbours 2, 4
        dist[2]=1, push               q = [2]
        dist[4]=1, push               q = [2,4]

pop 2   neighbours 1(done), 3, 4(done), 5
        dist[3]=2, push               q = [4,3]
        dist[5]=2, push               q = [4,3,5]

pop 4   neighbours 1(done), 2(done), 5(done)
                                      q = [3,5]

pop 3   neighbours 2(done), 6
```

```
        dist[6]=3, push                q = [5,6]

pop 5   neighbours 2(done), 4(done)    q = [6]

pop 6   neighbour 3(done)              q = []

order visited : 1 2 4 3 5 6
dist          : d[1]=0 d[2]=1 d[4]=1 d[3]=2 d[5]=2 d[6]=3
d[7], d[8]    : still -1, different component
```

Compare with the DFS order from Part 3, which was `1 2 3 6 4 5`. DFS reached vertex 6 second-to-last by diving; BFS reached it last because it is genuinely the farthest. DFS orders tell you nothing about distance.

Time O(n + m), space O(n). Same argument as DFS: every vertex enters the queue at most once, and each adjacency list is scanned once.

4.5 Reconstructing the path

Distance alone is often not enough. Record where each vertex was reached from.

```
vector<int> par(n + 1, 0);

// inside the loop, when you discover v:
dist[v] = dist[u] + 1;
par[v] = u;
q.push(v);

// afterwards, to print the path from s to t:
if (dist[t] == -1) {
    cout << -1 << "\n";                 // unreachable, this case is always tested
} else {
    vector<int> path;
    for (int cur = t; cur != 0; cur = par[cur]) path.push_back(cur);
    reverse(path.begin(), path.end());
    for (int v : path) cout << v << " ";
}
```

On G1, `par[6] = 3`, `par[3] = 2`, `par[2] = 1`, so the path 1 to 6 is `1 2 3 6`, of length 3, matching `dist[6]`.

Note `par[s] = 0` acts as the stop marker, which is why 1-indexing is convenient here.

4.6 BFS on a grid

The grid is a graph, so BFS is BFS. Only the neighbour loop changes.

```
int R, C;
vector<string> g;
```

```
int dx[4] = {-1, 1, 0, 0};
int dy[4] = { 0, 0,-1, 1};

vector<vector<int>> bfsGrid(int sx, int sy) {
    vector<vector<int>> d(R, vector<int>(C, -1));
    queue<pair<int,int>> q;

    d[sx][sy] = 0;
    q.push({sx, sy});

    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        for (int k = 0; k < 4; k++) {
            int nx = x + dx[k], ny = y + dy[k];
            if (nx < 0 || nx >= R || ny < 0 || ny >= C) continue;
            if (g[nx][ny] == '#') continue;
            if (d[nx][ny] != -1) continue;
            d[nx][ny] = d[x][y] + 1;
            q.push({nx, ny});
        }
    }
    return d;
}
```

On G4, starting from the top left cell:

```
grid                distances from (0,0)

. . # . .           0  1  #  7  8
. # # . .           1  #  #  6  7
. . . . .           2  3  4  5  6
# # . # .           #  #  5  #  7
```

The shortest route from the top left to the bottom right cell is 7 moves. Trace it: down, down, right, right, right, down, which is `(0,0) (1,0) (2,0) (2,1) (2,2) (2,3) (2,4) (3,4)`. Seven moves, and the distance grid agrees.

Notice the top right corner is at distance 8 even though it looks close. The wall at `(0,2)` forces the whole journey around the bottom. That is exactly the kind of thing BFS handles and eyeballing does not.

4.7 Multi-source BFS

Question: given several starting cells, what is the distance from each cell to the **nearest** start?

The naive answer is to run one BFS per source, which is O(k(n + m)). The right answer is to push **all** sources into the queue at the start with distance 0, and run a single BFS.

```
for (auto s : sources) {
    dist[s] = 0;
    q.push(s);
}
```

```
// then the identical loop
```

Why is this correct? Imagine adding one imaginary super source `S` joined to every real source by an edge of weight 0. BFS from `S` is exactly the loop above. Each vertex is labelled with its distance to the nearest real source, in a single $O(n + m)$ pass.

This appears constantly: “distance to the nearest 1 in a binary matrix”, “how long until fire reaches every cell”, “minimum time for any of the guards to reach each room”. Whenever you see the word **nearest** together with several starting points, this is the tool.

4.8 BFS from every component

```
for (int i = 1; i <= n; i++)
    if (dist[i] == -1)
        bfsFrom(i);
```

Same shape as the DFS loop. Distances from different components are not comparable, so this pattern is usually about components, not distances.

4.9 BFS or DFS

Question	Use
Fewest edges, unweighted	BFS
Distance to nearest of several sources	multi-source BFS
Level by level processing	BFS
Is there a path at all	either, DFS is shorter to write
Connected components	either
Cycle detection	either, DFS is more natural
Topological sort	either (Kahn is BFS, the other is DFS)
Bridges, articulation points, SCC	DFS, they need the DFS tree
Subtree information, tree DP	DFS
Graph is a chain of 10^6 vertices	BFS, no recursion depth problem
Weighted shortest path	neither, see Part 11

Rough rule: **BFS answers “how far”, DFS answers “what is the structure”.**

4.10 Common mistakes

- **Marking visited when popping instead of when pushing.** The same vertex enters the queue many times. On a dense graph this is a real time limit failure, and in variants that count things it is a wrong answer.
- **Using BFS on a weighted graph.** BFS counts edges, not weights. If the weights are not all equal, the answer is wrong. This is the single most common conceptual error in shortest path problems.

- **Forgetting the unreachable case.** `dist[t] == -1` must be handled and is always in the tests.
- **Using `vector` with `erase(begin())` as a queue.** That is O(n) per pop.
- **Reading `g[nx][ny]` before the bounds check.** C++ will happily read out of bounds.
- **Recomputing `q.size()` inside a loop condition** when doing level by level processing. Snapshot it, exactly as in the trees course.
- **Running a separate BFS per source** when multi-source BFS does it in one pass.
- **Pushing the source without setting `dist[s] = 0`.** Then the source looks unvisited and gets rediscovered.

4.11 Codeforces problems that test exactly this

- 601A, The Two Routes. BFS on a graph and on its complement. https://codeforces.com/problemset/problem/601/A
- 1130C, Connect. Two BFS runs plus a small brute force. https://codeforces.com/problemset/problem/1130/C
- 1365D, Solve The Maze. Multi-source reasoning on a grid. https://codeforces.com/problemset/problem/1365/D
- 796D, Police Stations. Multi-source BFS on a tree. https://codeforces.com/problemset/problem/796/D

4.12 Check yourself

1. State the BFS theorem, including its condition.
2. Why must `dist[v]` be set at push time rather than pop time?
3. Give the BFS distances from vertex 3 in G1.
4. Why does pushing all sources at the start compute the distance to the nearest source?
5. On G4, why is the top right cell at distance 8 rather than 4?
6. You need the shortest path itself, not just its length. What extra array do you keep, and how do you use it?
7. A problem has edge weights 1 and 5. Can you still use plain BFS? What would you do instead?

Part 5

Connected Components and Flood Fill

The first real application of DFS and BFS

5.1 The pattern

A **connected component** is a maximal set of vertices that can all reach each other. In G1 there are two: `{1,2,3,4,5,6}` and `{7,8}`.

The pattern for anything about components is four lines you already saw:

```
int comps = 0;
for (int i = 1; i <= n; i++) {
    if (!vis[i]) {
        comps++;
        dfs(i);          // or bfs(i)
    }
}
```

Every time the loop finds an unvisited vertex, that vertex belongs to a component nobody has touched yet, so we increment and explore all of it. When the loop ends, every vertex has been visited exactly once and `comps` is the answer.

Use DFS or BFS, it makes no difference to the answer. DFS is two lines shorter, BFS is safe on very deep graphs.

Time O(n + m). Space O(n).

5.2 Labelling every vertex with its component

Counting is rarely enough. Usually you want to know which component a vertex is in, and how big each one is.

```
vector<int> comp;          // comp[u] = component id of u, 0 means unassigned
vector<int> compSize;      // compSize[c] = number of vertices in component c

void dfs(int u, int c, vector<vector<int>>& adj) {
    comp[u] = c;
    compSize[c]++;
    for (int v : adj[u])
        if (comp[v] == 0)
            dfs(v, c, adj);
}
```

```
int main() {
    // ... read the graph ...
    comp.assign(n + 1, 0);
    compSize.assign(1, 0);          // index 0 unused

    int c = 0;
    for (int i = 1; i <= n; i++) {
        if (comp[i] == 0) {
            c++;
            compSize.push_back(0);
            dfs(i, c, adj);
        }
    }
}
```

Now these questions are all O(1) or O(n):

Are u and v connected?	comp[u] == comp[v]
How many components?	c
Size of the largest component?	max over compSize
Is the graph connected?	c == 1

On G1: `comp = [_,1,1,1,1,1,1,2,2]`, `compSize = [_,6,2]`, so the graph is not connected and the largest component has 6 vertices.

5.2.1 Two facts you will reuse

1. **To connect a graph with `c` components you need at least `c - 1` extra edges.** Join them in a chain. This is a very common one-line answer in Div 2 A and B problems.
2. **A component with `k` vertices and exactly `k - 1` edges is a tree.** More edges means it contains a cycle. Counting edges per component is a cheap way to detect that (Codeforces 977E is exactly this).

To count edges inside a component while doing DFS, add up the degrees of its vertices and divide by 2.

5.3 Flood fill on a grid

The grid version of the same pattern, and the reason the technique is called flood fill: you pour paint on a cell and it spreads to everything connected.

```
G4:
. . # . .
. # # . .
. . . . .
# # . # .
```

How many separate regions of `.` are there? Run flood fill from each unvisited free cell.

```
int R, C;
```

```
vector<string> g;
vector<vector<bool>> vis;
int dx[4] = {-1,1,0,0}, dy[4] = {0,0,-1,1};

int fill(int x, int y) {                  // returns the area of this region
    vis[x][y] = true;
    int area = 1;
    for (int k = 0; k < 4; k++) {
        int nx = x + dx[k], ny = y + dy[k];
        if (nx < 0 || nx >= R || ny < 0 || ny >= C) continue;
        if (g[nx][ny] == '#' || vis[nx][ny]) continue;
        area += fill(nx, ny);
    }
    return area;
}

int main() {
    // read R, C, g
    vis.assign(R, vector<bool>(C, false));
    int regions = 0, best = 0;
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            if (g[i][j] == '.' && !vis[i][j]) {
                regions++;
                best = max(best, fill(i, j));
            }
    cout << regions << " " << best << "\n";
}
```

G4 has 20 cells, 6 of them walls, so 14 free cells. Every free cell turns out to be reachable from every other, so the answer is one region of area 14.

Now turn cell `(2,2)` into a wall as well:

```
. . # . .
. # # . .
. . # . .
# # . # .
```

The board splits into three regions: the left block `{(0,0),(0,1),(1,0),(2,0),(2,1)}` of size 5, the right block of size 7, and the single cell `(3,2)`, which is now walled in on all four sides. Trace that by hand before moving on. Isolated cells are exactly the case people forget.

Complexity O(R * C), since each cell is entered once and does O(4) work. In graph terms `n = R*C` and `m <= 2*R*C`, so this is the usual O(n + m).

5.3.1 Recursion depth on grids

A 1000 by 1000 grid of free cells means up to 10^6 nested calls in the worst case (a snake-shaped corridor). That will overflow the stack on many judges. For large grids, write flood fill with an explicit BFS queue instead. The code is the grid BFS from Part 4 with the distance array replaced by a visited array. This is a real failure mode, not a theoretical one.

5.4 Variations you should be able to write immediately

`count components`	`counter around the loop`
`size of each component`	`accumulate during DFS`
`largest component`	`max of the sizes`
`are u and v connected`	`compare component ids`
`number of edges to make it connected`	`(number of components) - 1`
`count islands in a grid`	`flood fill over '1' cells`
`largest island area`	`flood fill returning area`
`colour each component differently`	`store comp[] and print it`
`smallest component containing vertex v`	`size of comp[v]`

For the "are u and v connected" question asked many times, and especially when edges are being **added** over time, Part 9 (DSU) is the better tool. DFS answers it once in O(n + m); DSU answers it in near O(1) per query and supports adding edges online.

5.5 Common mistakes

- **Starting DFS only from vertex 1.** The classic. Loop over all vertices.
- **Counting components on a directed graph with this method.** Plain DFS on a directed graph does not give components in any meaningful sense. What you want there is either "weakly connected" (ignore directions and use this method) or strongly connected components (Part 13). Read the problem carefully.
- **Forgetting that isolated vertices are components too.** A vertex with degree 0 is its own component, and problems test that.
- **Grid flood fill by recursion on a 10^6 cell grid.** Use BFS.
- **Marking `vis` only on the starting cell** and forgetting it inside the loop, so cells are pushed repeatedly.
- **Reading the grid with `cin >> g[i]` but forgetting to size the vector first.**

5.6 Codeforces problems that test exactly this

- 217A, Ice Skating. Components on points that share a row or column. https://codeforces.com/problemset/problem/217/A
- 977E, Cyclic Components. Component that is exactly a cycle: every vertex has degree 2. https://codeforces.com/problemset/problem/977/E
- 445B, DZY Loves Chemistry. Components, and the answer is a power of two. https://codeforces.com/problemset/problem/445/B
- 1176E, Cover it! Colour by BFS layer parity and pick the smaller class. https://codeforces.com/problemset/problem/1176/E

5.7 Check yourself

1. Why does the outer loop over all vertices give the correct component count?

2. G1 has two components. How many edges must be added to make it connected?
3. A component has 7 vertices and 7 edges. What can you say about it?
4. How do you compute the number of edges inside a component during a DFS?
5. Why is recursive flood fill dangerous on a 1000 by 1000 grid, and what do you do instead?
6. Two vertices are queried for connectivity 10^5 times, and edges keep getting added between queries. Is repeated DFS a good plan?
7. What is the component count of a graph with n vertices and no edges?

Part 6

Cycle Detection

Undirected and directed, and why they are different problems

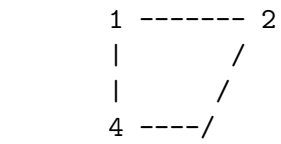
Cycles are the feature that separates graphs from trees, so detecting them is a basic skill. The undirected case and the directed case need different code, and using one for the other is a classic wrong answer.

6.1 Undirected graphs

6.1.1 The idea

Run a DFS. If you ever reach a vertex that is already visited **and is not the neighbour you came from**, you have found a second way to get there, which closes a cycle.

The "not the one you came from" clause exists because in an undirected graph every edge appears in both adjacency lists. Standing at 2 having come from 1, you will see 1 among your neighbours and it will be visited. That is not a cycle, that is the edge you just walked.




```
dfs(1) -> dfs(2): from 2, neighbour 1 is visited but it is the parent, ignore
       -> dfs(4) from 2: from 4, neighbour 1 is visited and is NOT the parent
          => cycle found: 1 - 2 - 4 - 1
```

6.1.2 Code

```cpp
bool dfsCycle(int u, int parent, vector<vector<int>>& adj, vector<bool>& vis) {
    vis[u] = true;
    for (int v : adj[u]) {
        if (!vis[v]) {
            if (dfsCycle(v, u, adj, vis)) return true;
        } else if (v != parent) {
            return true;                // visited, and not where we came from
        }
    }
    return false;
}

bool hasCycle(int n, vector<vector<int>>& adj) {
    vector<bool> vis(n + 1, false);
```

```
    for (int i = 1; i <= n; i++)
        if (!vis[i] && dfsCycle(i, 0, adj, vis))
            return true;
    return false;
}
```

`parent = 0` for the root of each DFS, which is why 1-indexing is convenient.

Time O(n + m), space O(n).

6.1.3 Dry run on G1

```
dfs(1, parent 0)
  vis={1}, neighbours 2, 4
  dfs(2, parent 1)
    vis={1,2}, neighbours 1, 3, 4, 5
    1 is visited and equals parent -> skip
    dfs(3, parent 2)
      neighbour 2 is visited and equals parent -> skip
      dfs(6, parent 3): neighbour 3 is parent -> skip, return false
      return false
    dfs(4, parent 2)
      neighbour 1: visited, and 1 != parent(2)   =>  CYCLE
```

The cycle is `1 - 2 - 4 - 1`, exactly the triangle in the picture.

6.1.4 The parent trick and its two traps

Multiple edges. If the input contains the edge `1-2` twice, then standing at 2 you see 1 twice. Both are "the parent", so the code above reports no cycle, when in fact two parallel edges *are* a cycle of length 2. If a problem allows multi-edges, compare **edge ids** instead of vertices: pass the index of the edge you arrived on and skip only that one.

Self loops. An edge `v-v` is a cycle by itself. The code above sees `v` as a visited non-parent neighbour and correctly reports a cycle, but some variants of the code skip `v == u`, so check your version.

If the statement says "the graph is simple", neither case can happen and the parent version is fine.

6.1.5 Printing the cycle

Keep a parent array and, when the back edge `u -> v` is found, walk up from `u` to `v`:

```
int cycleStart = -1, cycleEnd = -1;
vector<int> par;

bool dfs(int u, int p, vector<vector<int>>& adj, vector<bool>& vis) {
    vis[u] = true;
    par[u] = p;
    for (int v : adj[u]) {
        if (v == p) continue;
        if (vis[v]) { cycleStart = v; cycleEnd = u; return true; }
        if (dfs(v, u, adj, vis)) return true;
    }
    return false;
```

```
}

// afterwards:
vector<int> cycle;
cycle.push_back(cycleStart);
for (int v = cycleEnd; v != cycleStart; v = par[v]) cycle.push_back(v);
cycle.push_back(cycleStart);
reverse(cycle.begin(), cycle.end());
```

6.1.6 Counting your way out

For the single question "does this undirected graph contain a cycle", you do not even need DFS. A graph is a forest exactly when it has no cycle, and a forest with `n` vertices and `c` components has `n - c` edges. So:

```
cycle exists  <=>  m > n - c
```

Compute `c` with the component counting from Part 5. This is often the fastest thing to write, and it also gives you "how many extra edges beyond a forest".

The same fact applies per component: a component with `k` vertices and exactly `k` edges contains exactly one cycle.

6.1.7 The DSU method

Process every edge `(u, v)`. If `u` and `v` are already in the same set, this edge closes a cycle. Otherwise union them. That is Part 9, and it is the method to use when edges arrive one at a time.

6.2 Directed graphs

6.2.1 Why the undirected method fails

```
1 ---> 2
|      |
v      v
3 ---> 4
```

There is no directed cycle here: from 4 you cannot get back anywhere. But an undirected style DFS starting at 1 would go `1 -> 2 -> 4`, then from 1 go to 3, see 4 already visited and not the parent, and shout "cycle". Wrong.

The problem is that "already visited" is too coarse. What matters is **whether the vertex is still on the current recursion stack**, that is, whether we are still inside its call. An edge to a vertex that is finished is harmless. An edge to a vertex we are currently inside is a cycle.

6.2.2 Three colours

```
0 = white : not visited yet
1 = grey  : currently on the recursion stack, we entered it and have not left
2 = black : completely finished, we have left it
```

A **grey** neighbour means a back edge, which means a directed cycle. A black neighbour is fine.

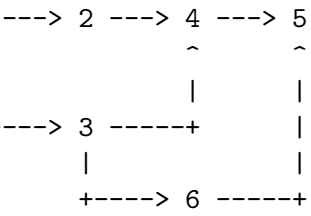
```cpp
vector<int> color;        // 0 white, 1 grey, 2 black

bool dfs(int u, vector<vector<int>>& adj) {
    color[u] = 1;                       // grey: entering
    for (int v : adj[u]) {
        if (color[v] == 1) return true;             // back edge -> cycle
        if (color[v] == 0 && dfs(v, adj)) return true;
    }
    color[u] = 2;                       // black: leaving
    return false;
}

bool hasDirectedCycle(int n, vector<vector<int>>& adj) {
    color.assign(n + 1, 0);
    for (int i = 1; i <= n; i++)
        if (color[i] == 0 && dfs(i, adj))
            return true;
    return false;
}
```

Notice there is no `parent` parameter at all. In a directed graph, an edge back to the vertex you came from is a genuine 2-cycle and must be reported.

6.2.3 Dry run on G2, which has no cycle



```
dfs(1): grey 1
  dfs(2): grey 2
    dfs(4): grey 4
      dfs(5): grey 5, no outgoing, black 5
      black 4
    black 2
  dfs(3): grey 3
    4 is black -> fine, not a cycle
    dfs(6): grey 6
      5 is black -> fine
      black 6
    black 3
  black 1

no cycle
```

That is the whole point: vertex 4 was visited twice as a target, and only the colour told us it was harmless.

Now add the edge `5 -> 1` and rerun. When we are inside `dfs(5)`, vertex 1 is grey, so we report a

cycle immediately, which is correct: `1 -> 2 -> 4 -> 5 -> 1`.

6.2.4 Printing the directed cycle

Same technique: keep `par[]`, and when you find a grey neighbour `v` from `u`, walk `u, par[u], par[par[u]], ...` back to `v`.

6.2.5 The Kahn alternative

Topological sorting (Part 8) gives you cycle detection for free: run Kahn's algorithm, and if fewer than `n` vertices come out, the leftovers form at least one cycle. Many people prefer this because it is iterative, so it cannot blow the stack, and it usually needs to be written anyway.

6.3 Summary table

	Undirected	Directed
Test	visited neighbour that is not the parent	neighbour that is grey
Extra state	parent vertex (or parent edge id)	3-colour array
Trap	multi-edges and self loops	using the parent trick, which is wrong
Alternative	DSU, or `m > n - c`	Kahn's algorithm counting output size
Complexity	O(n + m)	O(n + m)

6.4 Common mistakes

- **Using the parent trick on a directed graph.** Reports cycles that are not there.
- **Using the colour method on an undirected graph without handling the parent.** Every single edge looks like a 2-cycle, because `u -> v -> u` is always available.
- **Comparing parent vertices when the graph can have multiple edges.** Compare edge ids.
- **Only starting from vertex 1.** A cycle may live in another component.
- **Forgetting to set black on exit.** Then finished vertices stay grey and every cross edge looks like a cycle.
- **Returning as soon as one DFS finishes** instead of continuing the outer loop over all vertices.

6.5 Codeforces problems that test exactly this

- 977E, Cyclic Components. Undirected, degree based. https://codeforces.com/problemset/problem/977/E
- 510B, Fox And Two Dots. Cycle detection on a grid, with the parent rule. https://codeforces.com/problemset/problem/510/B
- 1249B2, Books Exchange. Cycles in a permutation, which is the simplest directed cycle structure. https://codeforces.com/problemset/problem/1249/B2
- For directed cycle detection in general, any topological sort problem from Part 8 works, because a failed topological sort is exactly a cycle.

6.6 Check yourself

1. Why do we skip the parent in the undirected version?
2. Give a graph where the parent trick wrongly reports “no cycle”, and say how to fix it.
3. Why is “already visited” not enough in a directed graph?
4. What do grey and black mean, precisely?
5. A graph has $n = 10$, $m = 12$ and 2 components. Does it contain a cycle? How do you know without running anything?
6. In the directed version, why is there no parent parameter?
7. How would you print the cycle rather than just report its existence?

Part 7

Bipartite Graphs

Two colouring and odd cycles

7.1 What bipartite means

A graph is **bipartite** if you can split the vertices into two groups so that **every edge goes between the groups**, and no edge stays inside a group.

Equivalently, and this is the version you use in code: you can colour every vertex black or white so that no edge joins two vertices of the same colour.

```
bipartite                       not bipartite
   1 --- 2                         1 --- 2
   |     |                          \   /
   3 --- 4                            3

colour 1,4 black                 the triangle needs 3 colours
colour 2,3 white
```

Real problems say it differently: split people into two teams so that no two enemies are on the same team; assign each task to one of two machines so that conflicting tasks differ; check whether a relation can be explained by two categories.

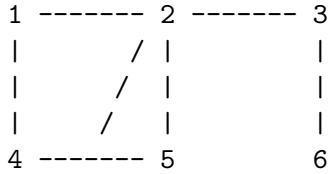
7.2 The theorem

> A graph is bipartite **if and only if** it contains no cycle of odd length.

The forward direction is easy to see. Walk along a cycle: the colour alternates at every step, so to come back to the starting vertex with the starting colour, the number of steps must be even.

The other direction is what the algorithm proves constructively: if you try to two-colour with BFS or DFS and never hit a conflict, the colouring you produced is a valid split.

For G1:



G1 contains the triangle `1 - 2 - 4 - 1`, which has length 3, odd. So G1 is **not** bipartite. No amount of cleverness will colour it.

Note the second component `{7, 8}` is bipartite on its own. A graph is bipartite only if **every** component is.

7.3 Two colouring by BFS

```cpp
// returns true if bipartite; colour[] holds 0/1 for each vertex, -1 if unvisited
bool isBipartite(int n, vector<vector<int>>& adj, vector<int>& color) {
    color.assign(n + 1, -1);

    for (int s = 1; s <= n; s++) {
        if (color[s] != -1) continue;        // already coloured, other component

        color[s] = 0;
        queue<int> q;
        q.push(s);

        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (color[v] == -1) {
                    color[v] = color[u] ^ 1;      // opposite colour
                    q.push(v);
                } else if (color[v] == color[u]) {
                    return false;                  // conflict, odd cycle
                }
            }
        }
    }
    return true;
}
```

`color[u] ^ 1` flips 0 to 1 and 1 to 0. You can also write `1 - color[u]`.

The outer loop matters: an uncoloured vertex means a new component, which must be started fresh with colour 0.

Time O(n + m), space O(n).

7.3.1 The DFS version

Identical logic, shorter:

```cpp
bool dfs(int u, int c, vector<vector<int>>& adj, vector<int>& color) {
    color[u] = c;
    for (int v : adj[u]) {
        if (color[v] == -1) {
            if (!dfs(v, c ^ 1, adj, color)) return false;
        } else if (color[v] == c) {
            return false;
        }
```

```
    }
    return true;
}
```

Use BFS if the graph might be a chain of 10^5 vertices, DFS otherwise. There is no difference in the answer.

7.3.2 Dry run on G1

```
start s = 1, color[1] = 0
pop 1: neighbours 2, 4
       color[2] = 1, push
       color[4] = 1, push
pop 2: neighbours 1(0, differs, ok), 3, 4, 5
       color[3] = 0, push
       vertex 4 already coloured 1, and color[2] is also 1  =>  CONFLICT
```

The conflict is on the edge `2–4`, which is exactly the edge that closes the odd cycle `1 - 2 - 4 - 1`. When a problem asks you to output the offending cycle, this edge plus the BFS parent pointers give it to you.

7.3.3 A bipartite example, traced

```
   1 --- 2
   |     |
   3 --- 4

adjacency: 1: 2 3    2: 1 4    3: 1 4    4: 2 3

color[1] = 0
pop 1: color[2] = 1, color[3] = 1
pop 2: 1 has colour 0, fine; color[4] = 0
pop 3: 1 fine; 4 already 0, and color[3] = 1, different, fine
pop 4: 2 and 3 both 1, and color[4] = 0, fine

bipartite. Side A = {1, 4}, side B = {2, 3}
```

The 4-cycle is even, so it colours fine. That is the theorem in one example.

7.4 Using the colouring afterwards

The colour array is usually more useful than the yes/no answer:

- The two sides are `{v : color[v] == 0}` and `{v : color[v] == 1}`. Codeforces 1176E asks you to output the smaller side, which is at most `n/2` vertices, and that is the entire problem.
- On a **tree**, two colouring is free and never fails, because a tree has no cycles at all. Colour by depth parity: even depth is side 0, odd depth is side 1.
- Bipartite graphs are where matching algorithms live. That is beyond this course, but knowing the word tells you which theory applies.

7.5 Common mistakes

- **Only running from vertex 1.** Other components may be non-bipartite, or may need their own starting colour.
- **Using a `visited` array separate from `color`.** Redundant. `color[v] == -1` means unvisited.
- **Checking the conflict only when discovering a new vertex.** The conflict shows up on an edge to an **already coloured** vertex, so the `else if` branch is where the answer is decided.
- **Believing that "no odd cycle" needs to be checked separately.** The colouring algorithm is the check.
- **Assuming a bipartite graph has equal sides.** It does not.
- **Forgetting that a graph with no edges is bipartite**, and so is a single vertex.

7.6 Codeforces problems that test exactly this

- 862B, Mahmoud and Ehab and the bipartiteness. Colour a tree, then count. https://codeforces.com/problemset/problem/862/B
- 1176E, Cover it! Take the smaller BFS parity class. https://codeforces.com/problemset/problem/1176/E
- 1144F, Graph Without Long Directed Paths. Orient edges using a 2-colouring. https://codeforces.com/problemset/problem/1144/F

7.7 Check yourself

1. State the odd cycle theorem in both directions.
2. Why is every tree bipartite?
3. Which edge of G1 causes the conflict, and which cycle does it close?
4. Why must the outer loop restart the colour at 0 for each component?
5. In the code, which branch actually detects non-bipartiteness?
6. A graph is bipartite with sides of size 3 and 7. What is the maximum number of edges it can have?
7. How would you output the odd cycle itself when the check fails?

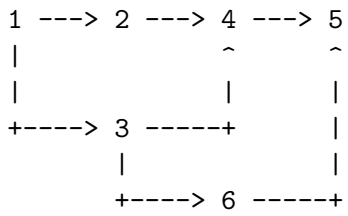
Part 8

Topological Sort and DP on a DAG

Ordering by dependency

8.1 The problem

You have tasks, and some must be done before others. Print an order in which all of them can be done.



That is G2. A valid order is `1 2 3 4 6 5`: whenever there is an edge `u -> v`, `u` appears before `v`.

A **topological order** is exactly that: a linear order of all vertices such that every directed edge points forward.

Two facts to internalise:

- It exists **if and only if** the graph is a DAG. A cycle `a -> b -> a` obviously cannot be linearised.
- It is usually **not unique**. `1 3 6 2 4 5` is also valid for G2. If a problem wants a specific one, it will say "lexicographically smallest", and that changes the algorithm slightly.

Undirected graphs do not have topological orders. This is a directed-only concept.

8.2 Kahn's algorithm, the BFS way

The idea in one sentence: **repeatedly output a vertex that has no remaining prerequisites.**

```
compute indegree of every vertex
put all vertices with indegree 0 into a queue
while the queue is not empty:
    pop u, append it to the order
    for each edge u -> v:
        indegree[v] -= 1                // u is done, one prerequisite less
        if indegree[v] == 0: push v
if the order has fewer than n vertices: the graph has a cycle
```

```cpp
vector<int> topoSort(int n, vector<vector<int>>& adj) {
    vector<int> indeg(n + 1, 0);
    for (int u = 1; u <= n; u++)
        for (int v : adj[u])
            indeg[v]++;

    queue<int> q;
    for (int i = 1; i <= n; i++)
        if (indeg[i] == 0) q.push(i);

    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);

        for (int v : adj[u]) {
            if (--indeg[v] == 0) q.push(v);
        }
    }

    if ((int)order.size() < n) return {};    // cycle: no topological order exists
    return order;
}
```

8.2.1 Dry run on G2

```
indegrees: 1:0  2:1  3:1  4:2  5:2  6:1
queue starts with all indegree 0 vertices: q = [1]

pop 1  order = [1]
       edge 1->2: indeg[2] 1->0, push
       edge 1->3: indeg[3] 1->0, push        q = [2,3]

pop 2  order = [1,2]
       edge 2->4: indeg[4] 2->1              q = [3]

pop 3  order = [1,2,3]
       edge 3->4: indeg[4] 1->0, push
       edge 3->6: indeg[6] 1->0, push        q = [4,6]

pop 4  order = [1,2,3,4]
       edge 4->5: indeg[5] 2->1              q = [6]

pop 6  order = [1,2,3,4,6]
       edge 6->5: indeg[5] 1->0, push        q = [5]

pop 5  order = [1,2,3,4,6,5]                 q = []

6 vertices produced out of 6, so no cycle.
```

Check the result against every edge: `1->2` ok, `1->3` ok, `2->4` ok, `3->4` ok, `3->6` ok, `4->5` ok, `6->5` ok.

Every arrow points to the right.

Time O(n + m), space O(n).

8.2.2 Cycle detection, free of charge

If the graph has a cycle, the vertices on that cycle can never reach indegree 0, because each of them is waiting for another one on the same cycle. So they never enter the queue and the output is short. `order.size() < n` is a complete and correct cycle test for directed graphs, and it is iterative, so there is no recursion depth risk.

This is often the easiest way to answer "is this directed graph acyclic".

8.3 The DFS way

Run a DFS. When a vertex **finishes** (all of its descendants are done), push it to a list. Reverse the list at the end.

```
vector<int> order;
vector<int> color;      // 0 white, 1 grey, 2 black

bool dfs(int u, vector<vector<int>>& adj) {
    color[u] = 1;
    for (int v : adj[u]) {
        if (color[v] == 1) return false;                  // cycle
        if (color[v] == 0 && !dfs(v, adj)) return false;
    }
    color[u] = 2;
    order.push_back(u);          // postorder position
    return true;
}

// after running dfs from every white vertex:
reverse(order.begin(), order.end());
```

Why does reversed postorder work? A vertex finishes only after everything reachable from it has finished. So in the postorder list, every descendant appears **before** its ancestor. Reverse it and every ancestor comes first, which is exactly the topological condition.

This is the same postorder position you used for bottom-up work in the trees course. Same idea, new use.

8.3.1 Dry run on G2

```
dfs(1) -> dfs(2) -> dfs(4) -> dfs(5), 5 has no outgoing, finish 5
                              finish 4
                    finish 2
       -> dfs(3): 4 is black, skip
                  dfs(6): 5 is black, skip, finish 6
                  finish 3
          finish 1
```

```
postorder : 5 4 2 6 3 1
reversed  : 1 3 6 2 4 5
```

A different valid order from Kahn's. Both are correct. If a checker expects one specific order, read the statement again.

8.4 Lexicographically smallest topological order

Replace the queue with a min-heap. At every step, among all vertices currently available, take the smallest.

```
priority_queue<int, vector<int>, greater<int>> pq;
// everything else is identical to Kahn
```

Time O(n + m log n).

This is a very common problem twist ("print the alphabetically first valid order"). Note that greedy is correct here only because any available vertex can legally be taken now, so taking the smallest never blocks anything.

Careful: this is **not** the same as "sort the topological order". You may only choose among vertices whose prerequisites are already satisfied.

8.5 DP on a DAG

This is where topological order earns its keep. A DAG has no cycles, so once you have a topological order, you can do dynamic programming in that order and every dependency is already computed.

The pattern:

```
process vertices in topological order
for each edge u -> v: update dp[v] using dp[u]
```

8.5.1 Counting paths

How many distinct paths lead from vertex 1 to vertex 5 in G2?

```
vector<long long> cnt(n + 1, 0);
cnt[1] = 1;
for (int u : order)
    for (int v : adj[u])
        cnt[v] += cnt[u];
```

```
order: 1 2 3 4 6 5

cnt[1] = 1
from 1: cnt[2] = 1, cnt[3] = 1
from 2: cnt[4] = 1
from 3: cnt[4] = 2, cnt[6] = 1
from 4: cnt[5] = 2
from 6: cnt[5] = 3
```

```
answer: 3 paths, namely 1-2-4-5, 1-3-4-5, 1-3-6-5
```

Use `long long`, or take everything modulo whatever the problem says. Path counts explode.

8.5.2 Longest path in a DAG

In a general graph the longest path problem is NP-hard. In a DAG it is a four line DP, and this contrast is worth remembering.

```
vector<int> dp(n + 1, 0);          // longest path ending at v, in edges
for (int u : order)
    for (int v : adj[u])
        dp[v] = max(dp[v], dp[u] + 1);
int answer = *max_element(dp.begin() + 1, dp.end());
```

On G2, starting the count from vertex 1:

```
dp[1]=0
dp[2]=1  dp[3]=1
dp[4]=max(dp[2]+1, dp[3]+1)=2
dp[6]=dp[3]+1=2
dp[5]=max(dp[4]+1, dp[6]+1)=3

longest path has 3 edges: 1-2-4-5 or 1-3-6-5 or 1-3-4-5
```

With weights, replace `+1` with `+w`. This also solves "longest increasing path in a grid" once you notice the grid becomes a DAG when you only allow moves to strictly larger cells.

8.5.3 Other things this pattern solves

shortest path in a DAG, even with negative weights	`dp[v] = min(dp[v], dp[u] + w)`
number of paths	`dp[v] += dp[u]`
can vertex t be reached from s	boolean dp
maximum profit picking a chain of tasks	weighted longest path
course scheduling feasibility	does a topological order exist
minimum number of semesters	longest path plus one

Notice that a DAG with negative weights is easy, while a general graph with negative weights needs Bellman-Ford. Cycles are what makes shortest paths hard, not negative numbers.

8.6 Common mistakes

- **Running topological sort on an undirected graph.** The concept does not apply.
- **Counting indegree by looping over `adj[v]`.** Indegree of `v` is the number of edges **into** `v`, so it is incremented while scanning `adj[u]` for all `u`.
- **Forgetting the `order.size() < n` check.** Then a cyclic input silently produces a partial order.
- **Using `--indeg[v] == 0` incorrectly**, for example writing `indeg[v]-- == 0`, which compares before decrementing.
- **Sorting the finished order to make it lexicographically smallest.** Wrong. Use a priority queue during the algorithm.

- **Doing the DP without a topological order.** The whole point is that the order guarantees dependencies are ready.
- **Overflow when counting paths.**

8.7 Codeforces problems that test exactly this

- 510C, Fox And Names. Build a graph from letter constraints, then topologically sort. https://codeforces.com/problemset/problem/510/C
- 999E, Reachability from the Capital. Uses the DAG of components, see Part 13. https://codeforces.com/problemset/problem/999/E
- 1100E, Andrew and Taxi. Binary search plus cycle detection on a directed graph. https://codeforces.com/problemset/problem/1100/E

8.8 Check yourself

1. When does a topological order exist?
2. Why does Kahn's algorithm produce fewer than n vertices exactly when there is a cycle?
3. Why is reversed DFS postorder a valid topological order?
4. Give a second valid topological order of G2, different from both in the text.
5. How do you get the lexicographically smallest order, and why is the greedy choice safe?
6. Why is longest path easy on a DAG but hard in general?
7. How many paths are there from vertex 1 to vertex 4 in G2? Verify with the DP.

Part 9

Disjoint Set Union

The competitive programmer's favourite data structure

9.1 The problem it solves

You have n items, initially all separate. Two operations arrive in any order, many times:

```
union(a, b)   put a and b into the same group
find(a)       which group is a in
```

and the real question is usually `find(a) == find(b)`, meaning "are a and b in the same group".

DFS answers connectivity **once**, in O(n + m). It cannot handle edges arriving over time, because every new edge would need a fresh traversal. DSU handles both operations in essentially constant time, which is why it appears in an enormous fraction of contest problems.

Other names for it: Union-Find, disjoint set forest.

9.2 The idea

Every group is a tree, and the **root** of that tree is the group's name. `find(x)` walks up to the root. `union(a, b)` hangs one root under the other.

```
initially                after union(1,2)         after union(1,4)

1  2  3  4  5             1      3  4  5           1        3  5
                          /                       / \
                         2                       2   4
```

The shape of these trees is meaningless. Only "who is the root" matters.

9.3 Version 1, the naive one

```
int parent[N];

void init(int n) { for (int i = 1; i <= n; i++) parent[i] = i; }

int find(int x) {
    while (parent[x] != x) x = parent[x];
    return x;
```

```
}

void unite(int a, int b) {
    a = find(a); b = find(b);
    if (a != b) parent[a] = b;
}
```

Correct, and potentially O(n) per operation. Union in a bad order builds a chain:

`union(1,2), union(2,3), union(3,4), ... produces`

```
1 -> 2 -> 3 -> 4 -> ... -> n
```

`and find(1) walks the entire chain.`

Two independent improvements fix this. Use both.

9.4 Improvement 1: path compression

While walking up in `find`, point every visited node straight at the root. The next `find` on any of them is then one step.

```
int find(int x) {
    if (parent[x] == x) return x;
    return parent[x] = find(parent[x]);       // compress on the way back up
}
```

That single line, `parent[x] = find(parent[x])`, does the whole job. It is a postorder assignment: recurse to the root first, then rewrite the pointer.

```
before find(4)            after find(4)

1                           1
|                          /|\
2                         2 3 4
|
3
|
4
```

9.5 Improvement 2: union by size

Always hang the **smaller** tree under the larger one. Then no tree ever gets deep, because a tree only gains depth when it is merged into something at least as big, which means the total size at least doubles.

```
int parent[N], sz[N];

void init(int n) {
```

```
    for (int i = 1; i <= n; i++) { parent[i] = i; sz[i] = 1; }
}

int find(int x) {
    if (parent[x] == x) return x;
    return parent[x] = find(parent[x]);
}

bool unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;          // already together, nothing to do
    if (sz[a] < sz[b]) swap(a, b);     // a is now the bigger root
    parent[b] = a;
    sz[a] += sz[b];
    return true;
}
```

That is the whole structure. Twenty lines, and you should be able to type it from memory in ninety seconds. `unite` returning a boolean is deliberate: `false` means "these were already connected", which is precisely the cycle test and precisely what Kruskal needs.

Union by **rank** (tree height) is an equivalent alternative. Size is more useful, because component sizes are often part of the answer.

9.6 Complexity

With both improvements, m operations on n elements cost **O(m alpha(n))**, where alpha is the inverse Ackermann function. For any n that fits in the universe, alpha(n) <= 4. So treat it as constant time and write O(m) or O(n + m) in your analysis.

With only path compression, or only union by size, it is O(log n) per operation, which is also fine in practice. With neither, it is O(n) per operation and it will fail.

Space O(n).

9.7 Dry run: G1 edge by edge

```
edges in input order: 1-2  1-4  2-3  2-4  2-5  3-6  4-5  7-8

start   parent = [1,2,3,4,5,6,7,8]   sizes all 1

1-2   find(1)=1, find(2)=2, different
      sizes equal, hang 2 under 1        parent[2]=1, sz[1]=2
1-4   find(1)=1, find(4)=4, different
      sz[1]=2 > sz[4]=1, hang 4 under 1  parent[4]=1, sz[1]=3
2-3   find(2)=1, find(3)=3, different    parent[3]=1, sz[1]=4
2-4   find(2)=1, find(4)=1, SAME         -> this edge closes a cycle
2-5   find(2)=1, find(5)=5, different    parent[5]=1, sz[1]=5
3-6   find(3)=1, find(6)=6, different    parent[6]=1, sz[1]=6
```

```
4-5   find(4)=1, find(5)=1, SAME          -> cycle again
7-8   different                           parent[8]=7, sz[7]=2

final groups: {1,2,3,4,5,6} with root 1, {7,8} with root 7
edges that closed a cycle: 2
```

Cross-check with the formula from Part 6: `m - (n - c) = 8 - (8 - 2) = 2`. The two counts agree, as they must.

9.8 What DSU is used for

connectivity queries, online	find(a) == find(b)
counting components	start at n, decrement on every successful unite
component sizes	sz[find(v)]
cycle detection while adding edges	unite returns false
Kruskal's MST	Part 10
grouping equal or equivalent items	"these two are the same", transitively
grid connectivity	id = x * C + y
offline queries in reverse	process deletions backwards as insertions

9.8.1 Counting components with DSU

```cpp
int comps = n;
// each time unite() actually merges:
if (unite(a, b)) comps--;
```

Starting from n singletons, every successful merge reduces the count by exactly one. This is the standard answer to "after each new road, how many separate regions remain".

9.8.2 The reverse trick

DSU can merge but cannot split. When a problem deletes edges or vertices over time, read all the queries first and process them **backwards**, so deletions become additions. This is a genuinely common technique and worth remembering as a shape, even before you meet a problem that needs it.

9.9 Common mistakes

- **Comparing `parent[a] == parent[b]` instead of `find(a) == find(b)`.** Parents are not roots. This is the number one DSU bug.
- **Uniting without calling `find` first.** `parent[a] = b` where a is not a root corrupts the structure.
- **Forgetting `init`**, or initialising `parent[i] = 0` instead of `i`.
- **Sizing the arrays n instead of n + 1** with 1-indexed vertices.
- **Not resetting between test cases.**
- **Using DSU on a directed graph** expecting it to respect directions. It cannot. DSU only knows undirected connectivity.
- **Recursion depth in `find`.** With path compression the trees stay shallow, so this is safe in practice, but if you skip union by size the recursion can be deep on adversarial input.
- **Expecting DSU to support splitting.** It does not.

9.10 Codeforces problems that test exactly this

- 217A, Ice Skating. https://codeforces.com/problemset/problem/217/A
- 445B, DZY Loves Chemistry. https://codeforces.com/problemset/problem/445/B
- 25D, Roads not only in Berland. Count redundant edges, reconnect components. https://codeforces.com/problemset/problem/25/D
- 455C, Civilization. DSU plus tree diameters. A very good exercise once you finish Part 12. https://codeforces.com/problemset/problem/455/C
- 500B, New Year Permutation. Positions that can be swapped form groups. https://codeforces.com/problemset/problem/500/B

9.11 Check yourself

1. Why is `parent[a] == parent[b]` the wrong test?
2. What are the two optimisations, and what does each one prevent?
3. Explain the line `return parent[x] = find(parent[x]);` in words.
4. What does it mean when `unite(a, b)` returns false, and which algorithm uses that?
5. Starting from n singletons, how do you maintain the number of components?
6. Why can DSU not undo a union, and what is the standard workaround?
7. G1 processed edge by edge produced two “already connected” events. Predict that number from n, m and the component count.

Part 10

Minimum Spanning Tree

Kruskal and Prim

10.1 The problem

Given a connected undirected weighted graph, choose a subset of edges that

1. keeps everything connected, and
2. has the smallest possible total weight.

Such a subset is always a **tree** on all n vertices: it has exactly `n - 1` edges and no cycles. If it had a cycle you could delete an edge of that cycle and stay connected while paying less. Hence the name **minimum spanning tree**, MST.

Typical phrasing in problems: connect all cities with roads at minimum cost; lay cable so every house has power; make the network connected as cheaply as possible.

If the graph is disconnected there is no spanning tree. Then the answer is usually a **minimum spanning forest**, one tree per component, and the problem will say so or will guarantee connectivity.

10.2 The working example, G3

```
edges (undirected, weighted):
1-2 : 7      1-3 : 9      1-6 : 14
2-3 : 10     2-4 : 15
3-4 : 11     3-6 : 2
4-5 : 6      5-6 : 9

n = 6, m = 9
```

The MST we are aiming for:

```
3-6 (2), 4-5 (6), 1-2 (7), 1-3 (9), 5-6 (9)      5 edges, total 33
```

10.3 The one idea behind both algorithms

The cut property. Split the vertices into any two non-empty groups. Look at all edges crossing between the groups. The cheapest one of those can always be part of some MST.

The intuition: the two groups must be joined somehow, so at least one crossing edge is in the tree. If your tree uses a more expensive crossing edge, swap it for the cheapest one. You stay connected and pay less.

Kruskal applies this globally by weight order. Prim applies it locally around a growing blob. That is the only real difference between them.

10.4 Kruskal's algorithm

```
sort all edges by weight, ascending
for each edge (u, v, w) in that order:
    if u and v are in different components:
        take the edge
        union their components
stop when you have n - 1 edges
```

The "different components" test is exactly `unite()` from Part 9, which is why Kruskal is taught right after DSU.

```cpp
struct Edge {
    int u, v, w;
    bool operator<(const Edge& o) const { return w < o.w; }
};

long long kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end());
    init(n);                                  // DSU init

    long long total = 0;
    int used = 0;

    for (const Edge& e : edges) {
        if (unite(e.u, e.v)) {                // false means it would make a cycle
            total += e.w;
            used++;
            if (used == n - 1) break;
        }
    }

    if (used < n - 1) return -1;              // graph was disconnected
    return total;
}
```

To output the edges themselves, push `e` into a vector inside the `if`.

10.4.1 Dry run on G3

```
sorted edges:
  3-6 (2)   4-5 (6)   1-2 (7)   1-3 (9)   5-6 (9)
  2-3 (10)  3-4 (11)  1-6 (14)  2-4 (15)

3-6 (2)   different -> take.   groups: {3,6} {1} {2} {4} {5}      total 2
```

```
4-5 (6)   different -> take.   groups: {3,6} {4,5} {1} {2}        total 8
1-2 (7)   different -> take.   groups: {1,2} {3,6} {4,5}          total 15
1-3 (9)   different -> take.   groups: {1,2,3,6} {4,5}            total 24
5-6 (9)   different -> take.   groups: {1,2,3,4,5,6}              total 33
                               5 edges = n - 1, stop

MST = { 3-6, 4-5, 1-2, 1-3, 5-6 },  weight 33
```

Notice that `2-3 (10)`, `3-4 (11)`, `1-6 (14)` and `2-4 (15)` were never even examined, because we stopped early. Also notice we would have rejected `2-3` anyway: by then 2 and 3 were already in the same group.

Time O(m log m) for the sort, plus O(m alpha(n)) for the DSU, so O(m log m) overall. Since `m <= n^2`, `log m <= 2 log n`, and you will also see this written as O(m log n). **Space O(n + m).**

10.5 Prim's algorithm

Grow one tree from a starting vertex. Repeatedly add the cheapest edge that leaves the current tree.

```
put vertex 1 in the tree
repeat n - 1 times:
    among all edges from the tree to outside, take the cheapest
    add its outside endpoint to the tree
```

To find "the cheapest edge leaving the tree" quickly, keep a priority queue of candidate edges.

```cpp
long long prim(int n, vector<vector<pair<int,int>>>& adj) {
    vector<bool> inTree(n + 1, false);
    priority_queue<pair<int,int>, vector<pair<int,int>>,
                   greater<pair<int,int>>> pq;      // (weight, vertex)

    long long total = 0;
    int taken = 0;
    pq.push({0, 1});                                // start at vertex 1, cost 0

    while (!pq.empty() && taken < n) {
        auto [w, u] = pq.top(); pq.pop();
        if (inTree[u]) continue;                    // stale entry, skip

        inTree[u] = true;
        total += w;
        taken++;

        for (auto [v, wt] : adj[u])
            if (!inTree[v])
                pq.push({wt, v});
    }

    if (taken < n) return -1;                       // disconnected
    return total;
}
```

Note the pair is `(weight, vertex)`, because the priority queue sorts by the first element. This is the opposite order from the adjacency list, which stores `(neighbour, weight)`. Mixing them up is a classic bug, so read your own code once before submitting.

10.5.1 Dry run on G3

```
start: tree = {1}, total = 0
       candidates from 1: (7,2) (9,3) (14,6)

take (7,2)   tree={1,2}      total=7
             add (10,3) (15,4)
             candidates: (9,3) (10,3) (14,6) (15,4)

take (9,3)   tree={1,2,3}    total=16
             add (11,4) (2,6)
             candidates: (2,6) (10,3 stale) (11,4) (14,6) (15,4)

take (2,6)   tree={1,2,3,6}  total=18
             add (9,5)
             candidates: (9,5) (11,4) (14,6 stale) (15,4)

take (9,5)   tree={1,2,3,5,6} total=27
             add (6,4)
             candidates: (6,4) (11,4) (15,4)

take (6,4)   tree=all         total=33

MST = { 1-2, 1-3, 3-6, 5-6, 4-5 },  weight 33
```

Same weight, same edge set as Kruskal. That is not luck: when all edge weights are distinct, the MST is unique. With ties, different algorithms may produce different trees of equal weight.

Time O(m log n) with a binary heap. **Space O(n + m).**

10.5.2 The dense version of Prim

If the graph is dense and given as a matrix, skip the heap: keep an array `minEdge[v]` and each round scan all vertices for the smallest. That is $O(n^2)$, which beats O(m log n) when m is near n^2.

```cpp
// O(n^2) Prim, for n <= about 5000 with an adjacency matrix
vector<int> minE(n + 1, INF);
vector<bool> used(n + 1, false);
minE[1] = 0;
long long total = 0;
for (int it = 0; it < n; it++) {
    int u = -1;
    for (int v = 1; v <= n; v++)
        if (!used[v] && (u == -1 || minE[v] < minE[u])) u = v;
    if (minE[u] == INF) return -1;                // disconnected
    used[u] = true;
    total += minE[u];
```

```
    for (int v = 1; v <= n; v++)
        if (!used[v]) minE[v] = min(minE[v], g[u][v]);
}
```

This version appears in problems where the graph is complete and the weight is a formula, for example "the cost between two points is their Manhattan distance". Building all n^2 edges for Kruskal would be too slow; $O(n^2)$ Prim is exactly right. Codeforces 1245D is the standard example.

10.6 Kruskal or Prim

	Kruskal	Prim (heap)	Prim (n^2)
Complexity	O(m log m)	O(m log n)	$O(n^2)$
Best when	sparse, edge list given	sparse, adjacency list given	dense, or complete graphs
Needs	DSU, sorted edges	priority queue	matrix
Handles disconnected input	yes, gives a forest	no, only reaches one component	no
Easier to write	usually yes	close	yes

Default to Kruskal. Switch to $O(n^2)$ Prim when the graph is complete or nearly so.

10.7 Useful variants

- **Maximum spanning tree**: sort descending, or negate the weights. Everything else is unchanged.
- **Minimum spanning forest**: run Kruskal and do not require n - 1 edges. The result has `n - c` edges.
- **Some edges are forced**: union them first, then run Kruskal on the rest.
- **Is the MST unique?** It is unique if, at every step, the choice was forced. A practical test: for each edge not in the MST, check whether it ties with the maximum edge weight on the tree path between its endpoints.
- **A virtual vertex**: problems like "you may either build a power plant in city i for cost c[i], or lay a cable between cities" become a plain MST if you add a vertex 0 joined to city i with weight c[i]. Then a spanning tree of the enlarged graph is exactly a valid plan. This trick is worth memorising, it shows up regularly.

10.8 Common mistakes

- **Forgetting to sort the edges.**
- **Using `find` results incorrectly**, that is, uniting non-roots.
- **Not detecting disconnection.** Check the edge count or the vertex count at the end.
- **Overflow.** With m up to 2 * 10^5 and weights up to 10^9, the total needs `long long`.
- **Mixing `(w, v)` and `(v, w)` orders in Prim.**
- **Forgetting the stale-entry check `if (inTree[u]) continue;` in heap Prim.**

- **Believing the MST minimises the distance between two specific vertices.** It does not. The path between two vertices in an MST is a **minimax** path (its largest edge is as small as possible), not a shortest path. Confusing MST with shortest paths is a real conceptual error.

10.9 Codeforces problems that test exactly this

- 1245D, Shichikuji and Power Grid. The virtual vertex trick, plus $O(n^2)$ Prim. https://codeforces.com/problemset/problem/1245/D
- 76A, Gift. MST with two weight functions. https://codeforces.com/problemset/problem/76/A
- 25D, Roads not only in Berland. Reuse redundant edges to connect components. https://codeforces.com/problemset/problem/25/D
- 1108F, MST Unification. Counting when an edge can be replaced. https://codeforces.com/problemset/problem/1108/F

10.10 Check yourself

1. Why does an MST always have exactly n - 1 edges?
2. State the cut property, and say how Kruskal and Prim each use it.
3. On G3, why was the edge `2–3 (10)` rejected by Kruskal?
4. When is the MST guaranteed unique?
5. Which Prim variant do you use on a complete graph with $n = 2000$, and why?
6. How do you compute a maximum spanning tree with the same code?
7. Is the path between u and v in the MST the shortest path between them? Justify your answer.

Part 11

Shortest Paths

BFS, 0-1 BFS, Dijkstra, Bellman-Ford, Floyd-Warshall

11.1 Choosing the algorithm

Do this before writing a single line. The choice is decided by the weights, not by the problem's story.

Situation	Algorithm	Complexity
All edges weight 1 (or unweighted)	BFS	O(n + m)
All weights are 0 or 1	0-1 BFS with a deque	O(n + m)
Weights small, say 0 to k	dial's buckets, or Dijkstra	O(n + m) k, or O(m log n)
Weights non-negative	Dijkstra	O(m log n)
Some weights negative	Bellman-Ford	O(n m)
Need distances between all pairs, n small	Floyd-Warshall	$O(n^3)$
The graph is a DAG	topological order DP (Part 8)	O(n + m)

Two sentences worth memorising:

- **Negative weights break Dijkstra.** Not "make it slow", break it.
- **Negative cycles make the question meaningless**, because you can loop forever and pay less each time. Bellman-Ford and Floyd can both detect them.

11.2 1. Unweighted: BFS

Covered in Part 4. One queue, `dist[v] = dist[u] + 1`, set at push time. O(n + m). If every edge costs the same amount `c`, run BFS and multiply by `c` at the end.

11.3 2. Weights 0 and 1: 0-1 BFS

Suppose every edge costs 0 or 1. Dijkstra would work but carries a log factor. There is a better way.

Idea: keep the queue sorted by distance without a heap. When you relax an edge of weight 0, the new vertex has the **same** distance as the current one, so it belongs at the **front**. When the edge has weight 1, it belongs at the **back**. A `deque` does both in O(1).

```cpp
vector<int> zeroOneBfs(int s, int n, vector<vector<pair<int,int>>>& adj) {
    vector<int> d(n + 1, INT_MAX);
    deque<int> dq;

    d[s] = 0;
    dq.push_front(s);

    while (!dq.empty()) {
        int u = dq.front(); dq.pop_front();

        for (auto [v, w] : adj[u]) {
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                if (w == 0) dq.push_front(v);
                else        dq.push_back(v);
            }
        }
    }
    return d;
}
```

The deque always holds at most two distinct distance values, `k` and `k + 1`, with all the `k`s in front. That invariant is exactly what a priority queue would maintain, achieved with no heap at all.

11.3.1 Dry run

```
edges (undirected, weight in brackets):
1-2 (0)   1-3 (1)   2-4 (1)   3-4 (0)   4-5 (0)

d[1]=0, dq = [1]

pop 1   edge (2,0): d[2]=0, push_front      dq = [2]
        edge (3,1): d[3]=1, push_back       dq = [2,3]

pop 2   edge (1,0): 0+0 not < 0, skip
        edge (4,1): d[4]=1, push_back       dq = [3,4]

pop 3   edge (1,1): no improvement
        edge (4,0): 1+0 = 1, not < 1, skip  dq = [4]

pop 4   edge (5,0): d[5]=1, push_front      dq = [5]

pop 5   done

d = [_, 0, 0, 1, 1, 1]
```

Where this appears: grids where moving in your current direction is free and turning costs 1; problems where you may delete or flip at most some cells; “minimum number of edges to reverse to make a path exist”, which becomes a 0-1 graph by adding each reversed edge with weight 1. Codeforces 1063B is the standard grid example.

11.4 3. Non-negative weights: Dijkstra

11.4.1 The idea

Keep a set of vertices whose distance is already final. Repeatedly take the unfinished vertex with the smallest tentative distance, declare it final, and relax its outgoing edges.

Why is the smallest tentative distance final? Because all weights are non-negative, so any other route to it would have to pass through some unfinished vertex first, which already has a larger tentative distance, and adding non-negative weights cannot make it smaller. **That argument fails the moment a weight is negative**, which is why Dijkstra requires non-negative weights.

11.4.2 The code

```
const long long INF = 1e18;

vector<long long> dijkstra(int s, int n, vector<vector<pair<int,int>>>& adj) {
    vector<long long> d(n + 1, INF);
    priority_queue<pair<long long,int>,
                   vector<pair<long long,int>>,
                   greater<pair<long long,int>>> pq;      // min-heap of (dist, vertex)

    d[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();

        if (du > d[u]) continue;            // stale entry, we already did better

        for (auto [v, w] : adj[u]) {
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                pq.push({d[v], v});         // lazy: push a new entry, never update in place
            }
        }
    }
    return d;
}
```

Three details that matter:

- **`(dist, vertex)` order** in the heap, because pairs compare on the first element.
- **The stale check** `if (du > d[u]) continue;`. We never remove outdated entries, we just ignore them. Without this line the algorithm still gives correct answers but does far more work.
- **`long long` for distances.** With 10^5 edges of weight 10^9, a path can reach 10^{14}. This is the single most common Dijkstra bug in contests.

Time O(m log n), because each edge can push at most one heap entry and each heap operation is O(log). **Space O(n + m).**

11.4.3 Dry run on G3

```
edges: 1-2:7  1-3:9  1-6:14  2-3:10  2-4:15  3-4:11  3-6:2  4-5:6  5-6:9

d = [0, INF, INF, INF, INF, INF]   (indices 1..6)
pq = {(0,1)}

pop (0,1)     relax 2: d[2]=7,  push (7,2)
              relax 3: d[3]=9,  push (9,3)
              relax 6: d[6]=14, push (14,6)

pop (7,2)     relax 3: 7+10=17 > 9, no
              relax 4: d[4]=22, push (22,4)

pop (9,3)     relax 4: 9+11=20 < 22, d[4]=20, push (20,4)
              relax 6: 9+2 =11 < 14, d[6]=11, push (11,6)

pop (11,6)    relax 5: 11+9=20, d[5]=20, push (20,5)

pop (14,6)    stale (14 > d[6]=11), skip

pop (20,4)    relax 5: 20+6=26 > 20, no

pop (20,5)    relax 4: 20+6=26 > 20, no

pop (22,4)    stale, skip

final: d[1]=0  d[2]=7  d[3]=9  d[4]=20  d[5]=20  d[6]=11
```

Trace the answer for vertex 5: `1 -> 3 -> 6 -> 5` costs `9 + 2 + 9 = 20`, which beats the more obvious `1 -> 2 -> 4 -> 5` costing `7 + 15 + 6 = 28`. This is exactly the kind of thing that greedy eyeballing gets wrong.

11.4.4 Why negative weights break it

```
1 -> 2 with weight 2
1 -> 3 with weight 5
3 -> 2 with weight -4

true shortest distance to 2 is 5 + (-4) = 1
```

Dijkstra pops vertex 2 at distance 2 and declares it final, before it has ever looked at vertex 3. Any implementation that marks vertices as settled returns 2, which is wrong. The lazy version above may recover on tiny cases by re-pushing, but on larger inputs it degenerates to exponential work. Do not use Dijkstra with negative weights, use Bellman-Ford.

11.4.5 Path reconstruction

```cpp
vector<int> par(n + 1, 0);
// inside the relaxation:
d[v] = d[u] + w;
```

```
par[v] = u;
pq.push({d[v], v});

// afterwards:
vector<int> path;
for (int cur = t; cur != 0; cur = par[cur]) path.push_back(cur);
reverse(path.begin(), path.end());
```

If `d[t] == INF`, print -1 or whatever the statement wants. Codeforces 20C is exactly this problem, and its tests include the unreachable case.

11.4.6 Variants worth knowing

- **Multi-source**: push every source with distance 0, exactly as in BFS.
- **Dijkstra on a grid**: the neighbour loop changes, nothing else. Common when moving into a cell has a cost.
- **Virtual vertex**: "you may start at any of these k cities" becomes one source vertex joined to those k with weight 0.
- **State expansion**: when the answer depends on more than the vertex, make the state bigger. `d[v][k]` for "at v having used k free tickets" is still Dijkstra, on a graph with n * (k+1) vertices. This is the single most useful advanced idea in this part, and Part 14 returns to it.
- **Counting shortest paths**: keep `cnt[v]`. When you improve `d[v]`, set `cnt[v] = cnt[u]`. When you tie, do `cnt[v] += cnt[u]`.

11.5 4. Negative weights: Bellman-Ford

11.5.1 The idea

A shortest path has at most `n - 1` edges (any more and it repeats a vertex, so it contains a cycle, which we may drop). So: relax **every** edge, `n - 1` times. After round `i`, all shortest paths using at most `i` edges are correct.

```
struct Edge { int u, v, w; };

vector<long long> bellmanFord(int s, int n, vector<Edge>& edges, bool& hasNegCycle) {
    vector<long long> d(n + 1, INF);
    d[s] = 0;

    for (int i = 0; i < n - 1; i++) {
        bool changed = false;
        for (const Edge& e : edges) {
            if (d[e.u] < INF && d[e.u] + e.w < d[e.v]) {
                d[e.v] = d[e.u] + e.w;
                changed = true;
            }
        }
        if (!changed) break;                 // early exit, common and worth having
    }

    hasNegCycle = false;
```

```
    for (const Edge& e : edges)              // one extra round
        if (d[e.u] < INF && d[e.u] + e.w < d[e.v])
            hasNegCycle = true;              // still improving means a negative cycle

    return d;
}
```

The guard `d[e.u] < INF` matters. Without it, `INF + (-5)` is smaller than `INF` and unreachable vertices start propagating nonsense.

11.5.2 Dry run

```
directed edges: 1->2 (4), 1->3 (5), 2->3 (-3), 3->4 (2)

d = [0, INF, INF, INF]

round 1: 1->2: d[2] = 4
         1->3: d[3] = 5
         2->3: 4 + (-3) = 1 < 5, d[3] = 1
         3->4: 1 + 2 = 3, d[4] = 3

round 2: nothing improves, early exit

d[1]=0  d[2]=4  d[3]=1  d[4]=3
```

Note the edge order was lucky here. With a different order it might take two or three rounds. That is why the algorithm runs `n - 1` of them.

11.5.3 Detecting a negative cycle

If any edge can still be relaxed after `n - 1` rounds, some path is getting shorter forever, which means a negative cycle is reachable from the source. To find **which vertices are affected**, run one more round and mark every vertex you improve, then propagate that mark along all outgoing edges with a BFS or DFS.

Time O(n m). Space O(n + m).

That complexity is why Bellman-Ford is the last resort. With n and m both 10^5 it is 10^{10} operations and hopeless. It is used when n is small, or when negative weights genuinely appear (currency arbitrage, profit and cost mixed together).

SPFA is a queue-based optimisation of Bellman-Ford that is usually fast but has a worst case of O(n m), and Codeforces problem setters have written anti-SPFA tests. Know it exists; do not rely on it.

11.6 5. All pairs: Floyd-Warshall

11.6.1 The idea

`d[i][j]` starts as the direct edge. Then, for each vertex `k` in turn, ask: “is it better to go from i to j via k?”

```
// d is an n+1 by n+1 matrix, d[i][i] = 0, d[i][j] = weight or INF
for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```

`k` must be the outermost loop. The meaning of the algorithm is "after processing k, `d[i][j]` is the best path using only vertices from `{1..k}` as intermediates". Put `k` inside and that invariant collapses and the answer is wrong. This is the most commonly memorised-wrong triple loop in all of competitive programming.

11.6.2 Dry run

```
directed: 1->2 (3), 1->3 (7), 2->3 (1), 2->4 (5), 3->4 (2)
(- means INF)

initial            after k=2            after k=3
     1 2 3 4            1 2 3 4              1 2 3 4
 1   0 3 7 -        1   0 3 4 8          1   0 3 4 6
 2   - 0 1 5        2   - 0 1 5          2   - 0 1 3
 3   - - 0 2        3   - - 0 2          3   - - 0 2
 4   - - - 0        4   - - - 0          4   - - - 0

k=1 changes nothing (no edges into 1)
k=2: d[1][3] = min(7, 3+1) = 4 ;  d[1][4] = min(-, 3+5) = 8
k=3: d[1][4] = min(8, 4+2) = 6 ;  d[2][4] = min(5, 1+2) = 3
k=4 changes nothing
```

Time $O(n^3)$, space $O(n^2)$. For $n = 500$ that is $1.25 * 10^8$ simple operations, which runs in well under a second. For $n = 1000$ it is 10^9 and usually too slow. So: $n <= 500$ comfortably, $n <= 800$ if the constant is tiny.

11.6.3 What else Floyd gives you

- **Negative cycle detection**: after the algorithm, `d[i][i] < 0` for some `i` means `i` sits on a negative cycle.
- **Transitive closure**: replace `min` and `+` with `or` and `and`, and you get "can i reach j" for all pairs. With `bitset<N>` rows this becomes $O(n^3 / 64)$, which is fast enough for n around 2000.
- **Handles negative edges** fine, as long as there is no negative cycle.
- **Path reconstruction**: keep `nxt[i][j]`, the first vertex after i on the best path, and update it whenever you update `d[i][j]`.

11.7 Comparison

Algorithm	Weights	Sources	Time	Detects negative cycles
BFS	all equal	one or many	O(n + m)	not applicable

Algorithm	Weights	Sources	Time	Detects negative cycles
0-1 BFS	0 and 1	one or many	O(n + m)	not applicable
Dijkstra	non-negative	one or many	O(m log n)	no, and it is wrong if they exist
DAG DP	any	one	O(n + m)	not applicable, a DAG has no cycles
Bellman-Ford	any	one	O(n m)	yes
Floyd-Warshall	any	all pairs	$O(n^3)$	yes, `d[i][i] < 0`

11.8 Common mistakes

- **Using `int` for distances.** Sums overflow. Use `long long` unless you have checked.
- **Setting `INF = INT_MAX` and then adding to it.** Overflow to negative. Use `1e18` with `long long`, or `1e9` with `int` and guard before adding.
- **Using BFS on a weighted graph.**
- **Using Dijkstra with negative weights.**
- **Omitting the stale check in Dijkstra.**
- **Putting `k` in the wrong loop position in Floyd.**
- **Forgetting `d[i][i] = 0` when initialising Floyd's matrix.**
- **Forgetting the unreachable case in the output.**
- **Forgetting that the graph may be directed** and adding both directions anyway.
- **Relaxing `INF` vertices in Bellman-Ford.**
- **Using `priority_queue` as a max-heap by accident.** The default is a max-heap; you need `greater<>` for a min-heap, or push negative distances.

11.9 Codeforces problems that test exactly this

- 20C, Dijkstra? Plain Dijkstra with path output. https://codeforces.com/problemset/problem/20/C
- 1063B, Labyrinth. Grid, 0-1 style reasoning about moves. https://codeforces.com/problemset/problem/1063/B
- 590C, Three States. Multi-source 0-1 BFS on a grid with three regions. https://codeforces.com/problemset/problem/590/C
- 449B, Jzzhu and Cities. Dijkstra plus counting shortest paths. https://codeforces.com/problemset/problem/449/B
- 295B, Greg and Graph. Floyd-Warshall run backwards. A beautiful exercise in why k is the outer loop. https://codeforces.com/problemset/problem/295/B
- 715B, Complete The Graph. A harder Dijkstra variant, for later. https://codeforces.com/problemset/problem/715/B

11.10 Check yourself

1. Which algorithm for weights in `{0, 1}`, and what data structure does it use?
2. Why is the smallest tentative distance final in Dijkstra, and where does that argument use non-negativity?
3. Write the Dijkstra stale check and explain what it prevents.
4. Give a three-vertex graph where Dijkstra returns the wrong answer.
5. Why does Bellman-Ford run exactly `n - 1` rounds?
6. How do you detect a negative cycle with Bellman-Ford? With Floyd?
7. Why must `k` be the outermost loop in Floyd-Warshall?
8. A problem has $n = 400$ vertices and 10^5 queries asking for the distance between two vertices. Which algorithm, and why not Dijkstra per query?
9. You must find the shortest path where you are allowed to make at most 2 edges free. What is the state, and which algorithm runs on it?

Part 12

Trees as Graphs

Everything from the trees course, in graph language

12.1 What changes and what does not

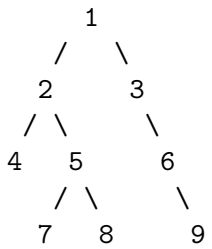
A **tree** is a connected undirected graph with no cycles. Equivalently: connected with exactly `n - 1` edges; or acyclic with exactly `n - 1` edges; or there is exactly one path between every pair of vertices.

In the binary trees course a node had `left` and `right` pointers and there was a root. In competitive programming a tree arrives as `n - 1` edges with no root and no ordering of children. So:

	Binary trees course	Trees in contests
Storage	`TreeNode*` with two pointers	`vector<int> adj[N]`
Root	given	you pick one, usually 1
Children	exactly `left` and `right`	any number, in input order
Going up	not possible	`par[]`, computed during DFS
Avoiding revisits	not needed	pass the parent, or use `vis`
Depth of recursion	O(height)	O(n), a tree can be a path

Everything conceptual transfers. `answer = combine(children answers, node)` is still the core pattern, and it is still computed in the postorder position. Only the loop over children changes.

12.2 The tree used in this part




```
edges: 1-2  1-3  2-4  2-5  3-6  5-7  5-8  6-9
n = 9, m = 8 = n - 1
```

Rooted at 1:

```
vertex : 1  2  3  4  5  6  7  8  9
depth  : 0  1  1  2  2  2  3  3  3
```

```
parent : 0  1  1  2  2  3  5  5  6
subsize: 9  5  3  1  3  2  1  1  1
```

Check the subtree sizes: `size(1) = 1 + size(2) + size(3) = 1 + 5 + 3 = 9`. Good.

12.3 Rooting a tree

```cpp
vector<int> par, dep, sz;

void dfs(int u, int p, vector<vector<int>>& adj) {
    par[u] = p;
    sz[u] = 1;

    for (int v : adj[u]) {
        if (v == p) continue;            // do not walk back into the parent
        dep[v] = dep[u] + 1;
        dfs(v, u, adj);
        sz[u] += sz[v];                  // postorder: children are done
    }
}

// call:  par.assign(n+1,0); dep.assign(n+1,0); sz.assign(n+1,0); dfs(1, 0, adj);
```

`if (v == p) continue;` replaces the visited array. In a tree it is enough, because the only way to revisit a vertex is through the edge you just came along. That is not true in a general graph, which is why Part 3 needed `vis`.

Note where each piece of work sits:

- `dep[v] = dep[u] + 1` is **before** the recursive call. Information flowing down, preorder position.
- `sz[u] += sz[v]` is **after** it. Information flowing up, postorder position.

Exactly the two positions from the trees course.

Time O(n), space O(n).

12.4 Subtree DP

Anything of the form “for every vertex, compute something about its subtree” is this same DFS with a different accumulation line:

```
subtree size            sz[u] += sz[v]
subtree sum of values   sum[u] += sum[v]
number of leaves        if u has no child, 1, else the sum over children
max value in subtree    best[u] = max(best[u], best[v])
count of white nodes    cnt[u] += cnt[v], plus 1 if u is white
depth of subtree        h[u] = max(h[u], h[v] + 1)
```

Codeforces 161D (“Distance in Tree”, count pairs at distance exactly k) is a good first non-trivial subtree DP, and 1092F is a good first **rerooting** problem, where you compute an answer for every possible root in O(n) total.

12.5 Tree diameter

The diameter is the number of edges on the longest path between any two vertices. In our tree it is 6, along `7 - 5 - 2 - 1 - 3 - 6 - 9`.

12.5.1 Method 1: two BFS runs

```
1. BFS from any vertex, find the farthest vertex a.
2. BFS from a, find the farthest vertex b.
3. dist(a, b) is the diameter.
```

```cpp
pair<int,int> bfsFar(int s, int n, vector<vector<int>>& adj) {
    vector<int> d(n + 1, -1);
    queue<int> q; d[s] = 0; q.push(s);
    int best = s;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (d[u] > d[best]) best = u;
        for (int v : adj[u]) if (d[v] == -1) { d[v] = d[u] + 1; q.push(v); }
    }
    return {best, d[best]};               // farthest vertex and its distance
}

int diameter(int n, vector<vector<int>>& adj) {
    int a = bfsFar(1, n, adj).first;
    return bfsFar(a, n, adj).second;
}
```

Why does it work? Because the farthest vertex from **any** starting vertex is always an endpoint of some diameter. Sketch: let the diameter be `a..b`. If the search from `s` ended at some `x` that is not an endpoint, you could splice the path `s..x` into the diameter and get something longer, contradiction. The full proof is a short case analysis and worth reading once.

12.5.2 Dry run

```
BFS from 1: distances 0,1,1,2,2,2,3,3,3. Farthest is 7 (or 8, or 9), pick 7.
BFS from 7: d[5]=1 d[2]=2 d[8]=2 d[4]=3 d[1]=3 d[3]=4 d[6]=5 d[9]=6
Farthest is 9 at distance 6.  Diameter = 6.
```

Time O(n). Only works on trees, not on general graphs. Do not try it on a graph with cycles.

12.5.3 Method 2: one DFS, bottom-up

This is the diameter algorithm from the trees course, now with any number of children.

```cpp
int best = 0;

int height(int u, int p, vector<vector<int>>& adj) {
    int top1 = 0, top2 = 0;               // two largest child heights
    for (int v : adj[u]) {
        if (v == p) continue;
        int h = height(v, u, adj) + 1;
        if (h > top1) { top2 = top1; top1 = h; }
```

```
        else if (h > top2) { top2 = h; }
    }
    best = max(best, top1 + top2);        // path turning at u
    return top1;                          // value for the parent
}
```

Note the same split as in the trees course: `top1 + top2` is recorded (the path that turns here), `top1` is returned (what the parent can extend). With two children this was `L + R` and `1 + max(L, R)`. With many children it becomes the two largest.

Use method 1 in a contest, it is shorter and hard to get wrong. Use method 2 when the tree is weighted or when you need extra information at every vertex.

12.6 LCA and distances

The **lowest common ancestor** of u and v, once the tree is rooted, is the deepest vertex that is an ancestor of both. In our tree, `LCA(4, 8) = 2` and `LCA(7, 9) = 1`.

The reason it matters:

`dist(u, v) = dep[u] + dep[v] - 2 * dep[LCA(u, v)]`

Check: `dist(4, 8) = 2 + 3 - 2 * dep[2] = 2 + 3 - 2 = 3`, and indeed the path is `4 - 2 - 5 - 8`.

Once you can compute LCA in O(log n), you can answer path queries between arbitrary vertices, which is the workhorse of tree problems.

12.6.1 Binary lifting

Precompute `up[v][k]`, the ancestor of `v` that is `2^k` steps above it.

```
up[v][0] = parent of v
up[v][k] = up[ up[v][k-1] ][k-1]        two jumps of 2^(k-1) make one of 2^k
```

```
const int LOG = 18;                       // 2^18 > 2*10^5
int up[N][LOG], dep[N];

void dfs(int u, int p, vector<vector<int>>& adj) {
    up[u][0] = p;
    for (int k = 1; k < LOG; k++)
        up[u][k] = up[ up[u][k-1] ][k-1];

    for (int v : adj[u])
        if (v != p) { dep[v] = dep[u] + 1; dfs(v, u, adj); }
}

int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);

    int diff = dep[u] - dep[v];
    for (int k = 0; k < LOG; k++)                 // lift u to v's depth
        if (diff >> k & 1) u = up[u][k];
```

```
    if (u == v) return u;

    for (int k = LOG - 1; k >= 0; k--)            // lift both as high as possible
        if (up[u][k] != up[v][k]) {               // without meeting
            u = up[u][k];
            v = up[v][k];
        }
    return up[u][0];                              // one more step, and they meet
}
```

Set `up[root][k] = 0` for all k, and make vertex 0 a self-parent sentinel so the jumps never fall off the top.

Preprocessing O(n log n), query O(log n), memory O(n log n). With $n = 2 * 10^5$ and $LOG = 18$ that is $3.6 * 10^6$ ints, about 14 MB. Acceptable, but be aware of it.

The second loop deserves a sentence, because it looks strange. We move up only when the two ancestors **differ**. If they are equal we might have jumped past the LCA. By going from the largest power down to the smallest, we end just below the LCA, and one final step from `up[u][0]` lands on it.

12.6.2 Dry run: LCA(4, 8)

```
dep[4] = 2, dep[8] = 3, so swap: u = 8, v = 4
diff = 1, lift u by 2^0:  u = up[8][0] = 5.   Now dep = 2 for both.
u = 5, v = 4, not equal.
k = 1: up[5][1] = 1, up[4][1] = 1, equal, do not move.
k = 0: up[5][0] = 2, up[4][0] = 2, equal, do not move.
answer = up[5][0] = 2
```

Correct: `LCA(4, 8) = 2`, and `dist = 2 + 3 - 2*1 = 3`.

12.6.3 Alternatives

- **Euler tour plus sparse table** gives O(1) queries after O(n log n) preprocessing. More code, rarely needed.
- **Offline Tarjan LCA with DSU** answers all queries in almost O(n + q). Elegant, but only when the queries are known in advance.
- For **small trees or few queries**, just walk up from the deeper vertex one step at a time. O(n) per query and ten lines of code, and it is the right answer when q is small.

12.7 Euler tour: subtree becomes a range

Record `tin[v]` and `tout[v]` exactly as in Part 3. Then the subtree of `v` occupies the contiguous range `[tin[v], tout[v]]` of the entry order.

```
in our tree, an Euler order from 1 is:
   1 2 4 5 7 8 3 6 9
positions:
   1 at 0, 2 at 1, 4 at 2, 5 at 3, 7 at 4, 8 at 5, 3 at 6, 6 at 7, 9 at 8
```

```
subtree of 2 = positions 1..5 = {2,4,5,7,8}   which is exactly sz[2] = 5 vertices
subtree of 3 = positions 6..8 = {3,6,9}
```

That turns "add x to every vertex in the subtree of v" and "sum the subtree of v" into range update and range query on an array, which a Fenwick tree or segment tree handles in O(log n). This is the standard bridge from trees to data structures, and it is worth knowing that it exists even before you learn segment trees.

12.8 Common mistakes

- **Forgetting `if (v == p) continue;`** and recursing back and forth forever.
- **Using the parent check on a graph that is not a tree.** It fails as soon as there is a cycle. Use `vis` there.
- **Applying the two BFS diameter trick to a general graph.** Trees only.
- **Assuming the tree is rooted at 1 when the problem roots it elsewhere.** Read the statement.
- **`LOG` too small.** With n = 2 * 10^5 you need at least 18.
- **Not handling `up[root][k]`.** Point it at a sentinel, usually 0.
- **Recursion depth on a path-shaped tree with n = 10^5.** This is the standard shape of an anti-recursion test.
- **Recomputing depth with a separate traversal** when the same DFS could have done it.

12.9 Codeforces problems that test exactly this

- 580C, Kefa and Park. Root the tree, carry a counter down. https://codeforces.com/problemset/problem/580/C
- 161D, Distance in Tree. Subtree DP with a small extra dimension. https://codeforces.com/problemset/problem/161/D
- 455C, Civilization. Diameters plus DSU merging. https://codeforces.com/problemset/problem/455/C
- 519E, A and B and Lecture Rooms. LCA plus subtree sizes. https://codeforces.com/problemset/problem/519/E
- 1092F, Tree with Maximum Cost. Rerooting. https://codeforces.com/problemset/problem/1092/F

12.10 Check yourself

1. Why does `if (v == p) continue;` suffice on a tree but not on a general graph?
2. Which line of the rooting DFS is preorder work, and which is postorder work?
3. Give the subtree sizes of every vertex in the example tree, and check them against n.
4. State the two-BFS diameter method and the reason it works.
5. Compute `dist(7, 9)` using depths and the LCA.
6. In binary lifting, why do we move only when `up[u][k] != up[v][k]`?
7. What does `tin` and `tout` buy you that a plain DFS does not?
8. Your LCA answers are wrong for deep vertices only. What is the first thing you check?

Part 13

The DFS Tree, Bridges and Strong Connectivity

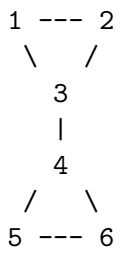
Advanced, but very much learnable

This part is marked advanced. Everything in it is one idea applied three times: **the DFS tree tells you which vertices can reach back upward, and that answers questions about what happens when you remove things.** Do not skip it out of fear. Do skip it if you have not yet solved twenty problems from the earlier parts.

13.1 Recap: the DFS tree

Run a DFS on an undirected graph. Tree edges are the ones DFS walks along. Every other edge is a **back edge**, joining a vertex to one of its own ancestors. There are no sideways edges (Part 3 explains why).

Working example for this part:




```
edges: 1-2  2-3  1-3  3-4  4-5  5-6  4-6
```

Two triangles joined by a single edge `3-4`. Intuitively, that edge is the weak point: cut it and the graph falls apart. Vertices 3 and 4 are also weak: delete either one and the graph falls apart. We want to find such edges and vertices automatically.

13.2 tin and low

Two numbers per vertex:

- `tin[v]`: the time at which DFS first entered `v`.
- `low[v]`: the smallest `tin` reachable from the subtree of `v` using tree edges downward and **at most one** back edge.

In words: `low[v]` answers "how high up can the subtree of v climb without using the edge to its parent?"

```cpp
int timer = 0;
vector<int> tin, low;
vector<bool> vis;

void dfs(int u, int p, vector<vector<int>>& adj) {
    vis[u] = true;
    tin[u] = low[u] = timer++;

    for (int v : adj[u]) {
        if (v == p) continue;                 // skip the edge to the parent
        if (vis[v]) {
            low[u] = min(low[u], tin[v]);     // back edge: climb to v
        } else {
            dfs(v, u, adj);
            low[u] = min(low[u], low[v]);     // child's reach becomes mine
        }
    }
}
```

Note carefully: for a back edge we take `tin[v]`, for a tree edge we take `low[v]`. Mixing these up is the standard bug, and it still gives correct answers on many small tests, which makes it hard to catch.

13.2.1 Dry run

```
dfs(1, 0):  tin[1] = low[1] = 0
  dfs(2, 1): tin[2] = low[2] = 1
    dfs(3, 2): tin[3] = low[3] = 2
      neighbour 1 is visited and is not the parent -> back edge, low[3] = min(2, tin[1]=0) = 0
      dfs(4, 3): tin[4] = low[4] = 3
        dfs(5, 4): tin[5] = low[5] = 4
          dfs(6, 5): tin[6] = low[6] = 5
            neighbour 4 is visited, not the parent -> low[6] = min(5, tin[4]=3) = 3
          back in 5: low[5] = min(4, low[6]=3) = 3
        back in 4: low[4] = min(3, low[5]=3) = 3
      back in 3: low[3] = min(0, low[4]=3) = 0
    back in 2: low[2] = min(1, low[3]=0) = 0
  back in 1: low[1] = min(0, low[2]=0) = 0

vertex : 1  2  3  4  5  6
tin    : 0  1  2  3  4  5
low    : 0  0  0  3  3  3
```

13.3 Bridges

A **bridge** is an edge whose removal increases the number of connected components. Also called a cut edge.

The tree edge `u - v` (with `v` a child of `u`) is a bridge **if and only if** `low[v] > tin[u]`.

Read it as: the subtree of `v` cannot reach `u` or anything above `u` except through this edge. So cutting it

isolates that subtree.

A back edge is never a bridge, because the cycle it closes provides an alternative route.

```
vector<pair<int,int>> bridges;

void dfs(int u, int p, vector<vector<int>>& adj) {
    vis[u] = true;
    tin[u] = low[u] = timer++;
    for (int v : adj[u]) {
        if (v == p) continue;
        if (vis[v]) {
            low[u] = min(low[u], tin[v]);
        } else {
            dfs(v, u, adj);
            low[u] = min(low[u], low[v]);
            if (low[v] > tin[u]) bridges.push_back({u, v});
        }
    }
}
```

On our example:

```
edge 1-2 : low[2]=0 > tin[1]=0 ?  no
edge 2-3 : low[3]=0 > tin[2]=1 ?  no
edge 3-4 : low[4]=3 > tin[3]=2 ?  YES, bridge
edge 4-5 : low[5]=3 > tin[4]=3 ?  no
edge 5-6 : low[6]=3 > tin[5]=4 ?  no

the only bridge is 3-4, exactly as the picture suggested
```

Time O(n + m), space O(n).

Multiple edges caveat: `if (v == p) continue;` skips **all** edges to the parent, so a double edge between u and v is wrongly treated as a single one, and the algorithm reports a bridge that is not one. If the input may have multi-edges, skip by **edge id** instead of by vertex.

13.4 Articulation points

An **articulation point** (cut vertex) is a vertex whose removal increases the number of components.

Two rules, because the root of the DFS is special:

- For a **non-root** vertex `u`: it is an articulation point if it has a child `v` with `low[v] >= tin[u]`. Note `>=`, not `>`. The subtree of `v` cannot get **above** `u`, so removing `u` cuts it off.
- For the **root**: it is an articulation point if and only if it has **more than one** child in the DFS tree. With two or more children, the root is the only thing joining them.

```
set<int> aps;

void dfs(int u, int p, vector<vector<int>>& adj) {
    vis[u] = true;
```

```
    tin[u] = low[u] = timer++;
    int children = 0;

    for (int v : adj[u]) {
        if (v == p) continue;
        if (vis[v]) {
            low[u] = min(low[u], tin[v]);
        } else {
            dfs(v, u, adj);
            low[u] = min(low[u], low[v]);
            if (low[v] >= tin[u] && p != 0) aps.insert(u);   // non-root rule
            children++;
        }
    }
    if (p == 0 && children > 1) aps.insert(u);               // root rule
}
```

On our example:

```
vertex 1 is the root, children in the DFS tree: just {2}  ->  not an AP
vertex 2: child 3, low[3]=0 >= tin[2]=1 ?  no   ->  not an AP
vertex 3: child 4, low[4]=3 >= tin[3]=2 ?  YES  ->  AP
vertex 4: child 5, low[5]=3 >= tin[4]=3 ?  YES  ->  AP
vertex 5: child 6, low[6]=3 >= tin[5]=4 ?  no   ->  not an AP
vertex 6: no children                            ->  not an AP

articulation points: 3 and 4
```

Which matches the picture: deleting 3 separates `{1,2}` from the rest, deleting 4 separates `{5,6}`.

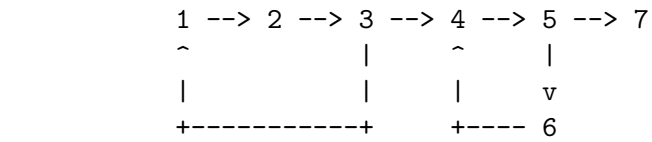
Why >= here and > for bridges? For a bridge we need the subtree to be unable to reach `u` **at all**. For an articulation point it is enough that the subtree cannot get **strictly above** `u`, because `u` itself is being deleted.

13.5 Strongly connected components

Now directed graphs. In a directed graph, "connected" splits in two:

- **Weakly connected**: connected if you ignore the directions.
- **Strongly connected**: every vertex can reach every other vertex, following the directions.

A **strongly connected component** (SCC) is a maximal set of vertices that can all reach each other.




```
edges: 1->2  2->3  3->1  3->4  4->5  5->6  6->4  5->7

SCCs: {1,2,3}   {4,5,6}   {7}
```

13.5.1 Kosaraju's algorithm

Two DFS passes, and it is the easiest SCC algorithm to remember.

1. Run DFS on the graph. Push each vertex onto a list when it FINISHES.
2. Reverse every edge.
3. Process vertices in reverse finish order. Each DFS on the reversed graph that starts from an unvisited vertex marks exactly one SCC.

```
vector<vector<int>> adj, radj;
vector<bool> vis;
vector<int> order, comp;

void dfs1(int u) {
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) dfs1(v);
    order.push_back(u);                          // finish time order
}

void dfs2(int u, int c) {
    comp[u] = c;
    for (int v : radj[u]) if (comp[v] == -1) dfs2(v, c);
}

int kosaraju(int n) {
    vis.assign(n + 1, false);
    order.clear();
    for (int i = 1; i <= n; i++) if (!vis[i]) dfs1(i);

    comp.assign(n + 1, -1);
    int c = 0;
    for (int i = n - 1; i >= 0; i--) {           // reverse finish order
        int u = order[i];
        if (comp[u] == -1) dfs2(u, c++);
    }
    return c;                                    // number of SCCs
}
```

Build `radj` while reading the input: for every edge `u -> v`, push `v` into `adj[u]` and `u` into `radj[v]`.

Why does it work? Intuition rather than proof: the vertex that finishes last in the first pass lies in a "source" SCC of the condensation, one with no incoming edges from other components. Starting a DFS from it in the **reversed** graph can only reach vertices of its own SCC, because the reversed edges to other components point away. Peel that component off and repeat.

Time O(n + m), space O(n + m).

Tarjan's algorithm does the same in a single pass using `low` values, similar in spirit to bridges. It is faster by a constant factor and harder to recall under pressure. Learn Kosaraju first.

13.5.2 The condensation graph

Shrink each SCC to a single vertex. The result is **always a DAG**, because any cycle among components would merge them into one component.

`condensation of the example:`

```
[1,2,3] ---> [4,5,6] ---> [7]
```

This is enormously useful. Any question on a directed graph that is hard because of cycles becomes a DAG question, and DAGs support topological sorting and DP (Part 8).

Typical uses:

- "How many extra edges to make the whole graph strongly connected?" Build the condensation, count components with indegree 0 (call it `a`) and outdegree 0 (call it `b`). The answer is `max(a, b)`, or 0 if there is only one component.
- "From how few vertices can we reach everything?" Count the condensation components with indegree 0. That is Codeforces 999E almost exactly.
- Longest path in a directed graph with cycles: condense, then DP on the DAG.

```
// building the condensation
for (int u = 1; u <= n; u++)
    for (int v : adj[u])
        if (comp[u] != comp[v])
            cadj[comp[u]].push_back(comp[v]);      // deduplicate if needed
```

13.6 Common mistakes

- **Using `low[v]` instead of `tin[v]` on a back edge.** The classic.
- **Using > instead of >= for articulation points**, or the reverse for bridges.
- **Forgetting the special rule for the root.**
- **Skipping the parent by vertex when multi-edges exist.** Skip by edge index.
- **Forgetting to run the outer loop over all vertices** on a disconnected graph.
- **Building the reversed graph incorrectly in Kosaraju**, most often by reversing only some edges.
- **Assuming the condensation might have a cycle.** It cannot.
- **Recursion depth.** These are all DFS, so a 2 * 10^5 chain is a real risk.

13.7 Codeforces problems that test exactly this

- 118E, Bertown roads. Orient every edge so the graph stays strongly connected. Possible exactly when there is no bridge. https://codeforces.com/problemset/problem/118/E
- 1000E, We Need More Bosses. Bridges, condensation into a tree, then tree diameter. A perfect combination of Parts 12 and 13. https://codeforces.com/problemset/problem/1000/E
- 427C, Checkposts. SCCs, and count the cheapest vertex in each. https://codeforces.com/problemset/problem/427/C
- 999E, Reachability from the Capital. Condensation and indegree-zero counting. https://codeforces.com/problemset/problem/999/E

13.8 Check yourself

1. Define `low[v]` in one sentence.

2. Why do we use `tin[v]` for a back edge and `low[v]` for a tree edge?
3. Why is a back edge never a bridge?
4. State the bridge condition and the articulation point condition, and explain the difference between `>` and `>=`.
5. Why does the root need its own rule?
6. What is the difference between weakly and strongly connected?
7. Why is the condensation always a DAG?
8. A directed graph has SCC condensation with 5 components, 2 of indegree 0 and 3 of outdegree 0. How many edges must be added to make it strongly connected?

Part 14

How to Think About Graph Problems

Modelling first, algorithm second

This is the most important part of the course. Everything before it was equipment.

In a contest, the hard part is almost never "implement Dijkstra". You can do that. The hard part is noticing that the problem **is** a shortest path problem, and working out what the vertices are.

14.1 The ten questions

Ask these in order, on any problem that smells like a graph.

1. What are the vertices? Cities, cells, people, states of a process, positions plus extra information. Write it down explicitly. If you cannot name the vertex set in one sentence, you have not modelled the problem yet.

2. What are the edges, and what does one edge mean? "An edge means I can move from A to B in one step." One step of what?

3. Directed or undirected? Look for the word "one-way", or an asymmetric relation. When in doubt, ask whether the relation is symmetric.

4. Weighted or unweighted? Are all moves equally expensive? If yes, BFS. If the costs are 0 and 1, deque. Otherwise Dijkstra.

5. Are there cycles? If the relation is "must happen before", the graph should be a DAG, and a cycle means the answer is "impossible". That is often the entire problem.

6. What is being asked: reachability, distance, ordering, connectivity, or structure?

```
reachability   -> DFS or BFS
distance       -> BFS, 0-1 BFS, Dijkstra, Bellman-Ford, Floyd
ordering       -> topological sort
connectivity   -> components, DSU
structure      -> bridges, articulation points, SCC, MST
```

7. How big is the graph? `n` and `m` decide whether $O(n^3)$, $O(m \log n)$ or $O(n + m)$ is allowed. And if the vertices are states, count the states before you build anything.

8. Is the graph explicit or implicit? An explicit graph is given as an edge list. An implicit graph (grid, states, `i -> a[i]`) is generated on demand. Most of the hard modelling problems are implicit.

9. Does the state need more than the vertex? "Shortest path, but you may make up to k edges free" is not a graph on n vertices. It is a graph on `n * (k+1)` states. This is the single most valuable idea in this part.

10. Is there a transformation that makes the problem standard? Reverse the edges. Add a super source. Split a vertex into two. Take the complement. Condense the SCCs. Each of these turns a strange problem into a textbook one.

14.2 The modelling catalogue

14.2.1 1. Explicit graph

The statement gives n, m and the edges. Read it in, pick an algorithm. Nothing to model.

14.2.2 2. Grid

Cells are vertices, adjacent free cells are edges. Never build the adjacency list, use the `dx/dy` loop (Part 2).

Watch for the variations: walls, teleporters (extra edges), different costs per cell (Dijkstra), moves that change direction (state expansion, see below).

14.2.3 3. State graph

The vertices are configurations, the edges are legal moves. The classic examples:

```
"minimum button presses to turn number x into y"
    vertex = current number
    edges  = the operations, each costing 1
    -> BFS

"minimum moves of a sliding puzzle"
    vertex = the whole board configuration
    edges  = sliding one tile
    -> BFS, with a map<string,int> for distances

"can you measure exactly 4 litres with jugs of 3 and 5"
    vertex = (amount in jug A, amount in jug B)
    edges  = fill, empty, pour
    -> BFS
```

The test for this family: the problem asks for the **minimum number of operations**, and each operation transforms one situation into another. That is BFS on states, always.

Before coding, **count the states**. Jugs of size 100 give 101 * 101 = 10201 states, which is nothing. A board of 20 items with a permutation state gives 20 factorial, which is impossible. The state count is your feasibility check.

14.2.4 4. State expansion

The answer depends on the vertex **and** something else. Put the something else into the vertex.

```
"shortest path, you may use at most k free edges"
    state = (vertex, number of free edges used)
    size  = n * (k + 1)
```

```
"shortest path, but you must arrive with an even number of steps"
    state = (vertex, parity)
    size  = 2n

"shortest path on a grid where turning costs 1 and going straight is free"
    state = (cell, current direction)
    size  = 4 * R * C, and it is a 0-1 BFS

"cheapest route with at most one refuel"
    state = (city, fuel remaining)
```

The algorithm does not change. Dijkstra on the expanded graph is still Dijkstra. Only your `d` array grows an extra dimension: `d[v][k]` instead of `d[v]`.

14.2.5 5. Node splitting

A constraint on a **vertex** (a capacity, a cost to pass through, a limit on visits) cannot be expressed with edge weights. Split the vertex into `v_in` and `v_out`, joined by an edge carrying the vertex's property. All original incoming edges go to `v_in`, all outgoing edges leave `v_out`.

14.2.6 6. Super source and super sink

"You may start from any of these k vertices" becomes: add vertex 0, join it to each of the k with weight 0, run a single search from 0. Same for "you may finish at any of these".

The MST version of this trick, where a vertex represents "build your own power plant", was in Part 10.

14.2.7 7. Reverse the graph

"What is the distance from every vertex to vertex t" is not a single-source problem, until you reverse every edge and run a single search from `t`. This turns an n-times problem into a once problem. Reach for it whenever the target is fixed and the source varies.

14.2.8 8. Complement graph

Sometimes the graph you need is the complement of the given one, and the given one is dense while the complement is sparse. There is a standard technique using a set of unvisited vertices during BFS, which keeps the total work at $O(n + m)$. Know it exists.

14.2.9 9. Dependencies become a DAG

"A must come before B", "you must learn X before Y", "the letters are in some unknown alphabet order". Build an edge for each constraint and topologically sort. If the sort fails, the constraints contradict each other, and reporting that is usually the point of the problem. Codeforces 510C is the model example.

14.2.10 10. Equivalence becomes DSU

"These two are interchangeable", "swap positions i and j freely", "these accounts belong to the same person". Union them all, then work per group. The moment a problem says "and this relation is transitive", it is DSU.

14.2.11 11. Two options per item becomes 2-colouring

"Each person joins one of two teams and enemies must be separated" is bipartiteness. If the constraints get more complicated ("if A is in team 1 then B must be in team 2"), you are looking at 2-SAT, which is implication edges plus SCC. That is beyond this course, but Part 13 gave you its machinery.

14.2.12 12. `i -> a[i]` is a functional graph

An array of pointers, a permutation, "the winner of match i plays match a[i]". Every vertex has outdegree 1, so the structure is always a set of cycles with trees hanging off them. Questions about "eventually you come back" are about the cycle.

14.3 The decision guide

```
Is the answer "minimum number of moves" and every move costs the same?
    yes -> BFS  (expand the state if the answer needs more than a position)

Are there weights?
    0/1        -> deque BFS
    non-negative -> Dijkstra
    negative     -> Bellman-Ford
    all pairs, n small -> Floyd

Is it "who must come before whom", or "is this possible without contradiction"?
    -> topological sort on a DAG, and a failed sort means a cycle

Is it "are these two in the same group", especially with edges arriving over time?
    -> DSU

Is it "connect everything as cheaply as possible"?
    -> MST

Is it "which single edge or vertex breaks the graph"?
    -> bridges and articulation points

Is it a directed graph where cycles are getting in the way?
    -> SCC, then work on the condensation DAG

Is it a tree, and the question is about paths or subtrees?
    -> root it, subtree DP, LCA
```

14.4 Pattern to problem map

```
Pattern                    Signature in the statement          Algorithm
-------------------------------------------------------------------------------
Reachability               "can you get from a to b"           DFS or BFS
Components                 "how many groups / regions"         DFS or DSU
```

Shortest unweighted	"minimum number of moves"	BFS
Nearest of many sources	"distance to the closest X"	multi-source BFS
Shortest weighted	"minimum total cost"	Dijkstra
Negative costs	"you may gain money"	Bellman-Ford
All pairs	"q queries about any two"	Floyd, n small
Ordering	"must be done before"	topological sort
Feasibility of order	"is it possible"	topological sort fails
Cheapest connection	"connect all cities"	MST
Dynamic connectivity	"after each new road"	DSU
Critical edges	"which road, if closed"	bridges
Critical vertices	"which city, if destroyed"	articulation points
Directed connectivity	"everyone can reach everyone"	SCC
Tree paths	"distance between two nodes"	LCA
Tree subtrees	"for each subtree"	subtree DP
State search	"minimum operations"	BFS on states
Extra dimension	"at most k times you may"	state expansion

14.5 Three worked examples

14.5.1 Example A

> "There are n cities and m one-way roads. You start in city 1. Find the minimum number of roads you must reverse so that city n becomes reachable."

- Vertices: cities. Edges: roads.
- Directed. Weighted? Not stated, but reversing costs something.
- **The move**: use a road forwards for free, or backwards at a cost of 1.
- So build **two** edges per road: `u -> v` with weight 0, and `v -> u` with weight 1.
- Weights are 0 and 1, so this is 0-1 BFS. O(n + m).

Question 2 ("what does one edge mean") did all the work.

14.5.2 Example B

> "A grid with walls. You may break at most k walls. Find the minimum number of steps from the top left to the bottom right."

- The vertex is not a cell. Two visits to the same cell are different if you have broken different numbers of walls.
- State: `(row, column, walls broken so far)`. Size `R * C * (k + 1)`.
- Every move costs 1, so BFS on states.
- `dist[R][C][k+1]`, and moving into a wall increases the third coordinate.

Question 9 did all the work.

14.5.3 Example C

> "n people, and m statements of the form 'i and j are the same species'. Then q queries asking whether two people are the same species."

- No distances, no ordering. Just “same group”, and the relation is transitive.
- DSU. Union each statement, answer each query with `find(a) == find(b)`.
- If the statements could also say “i and j are **different** species, and there are exactly two species”, it becomes bipartite checking, or DSU with parity.

Question 6 did all the work.

14.6 When you are stuck

In order:

1. **Draw a small case, 5 or 6 vertices, and solve it by hand.** Then ask what you did.
2. **Say the vertex set out loud.** “A vertex is a cell together with the direction I am facing.” If you cannot finish that sentence, that is the blocker.
3. **Count the states.** If it is 10^{12}, your model is wrong, not your code.
4. **Look at the constraints again.** n <= 500 with q queries screams Floyd. n <= 20 screams bitmask. m <= 2*10^5 screams O(m log n).
5. **Check whether a transformation makes it standard**: reverse, super source, split, condense.
6. **Ask what makes it non-standard**, and put that thing into the state.

14.7 Habits of strong contestants

- They write the vertex definition as a comment before writing code.
- They compute the state count and compare it with the time limit before implementing.
- They test on: a disconnected graph, a single vertex, a self loop if allowed, and the largest n.
- They use `long long` for any sum of weights, by default.
- They keep a personal template file with DSU, Dijkstra, BFS and the DFS skeletons, so implementation time goes to the modelling.
- They read the constraints before the statement.

14.8 Check yourself

1. A problem asks for the minimum number of operations to turn one string into another using a fixed set of moves. What are the vertices, and which algorithm?
2. You may use a “teleport” at most twice on a shortest path. How does the state change?
3. The problem asks for the distance from every city to the capital. Why reverse the edges?
4. A statement says “each of the m constraints says task a must precede task b, report any valid schedule or say it is impossible”. Name the algorithm and the failure test.
5. Costs are 0 and 1. Why not just use Dijkstra? What do you gain from the deque?
6. When does “same group and transitive” not mean DSU?
7. You modelled a problem and the state count is 10^{11}. What do you do next?

Part 15

Practice Problem Sets

Graded Codeforces problems, with hints and no solutions

15.1 How to use this set

1. **Answer the "think about" line before writing code.** That line is the exercise. The typing is not.
2. **Time yourself**: 25 minutes for a Level 1 or 2 problem, 45 for Level 4. If you are stuck past that, read one hint, not an editorial.
3. **After solving, name the pattern out loud.** "That was multi-source BFS." Naming is what makes the next one faster.
4. **Always test**: a disconnected graph, n = 1, the maximum n, and self loops if the statement allows them.
5. **Redo, do not collect.** Twenty problems solved twice beats forty solved once.

Difficulty ratings below are approximate. Use them as a rough ladder, not as gospel.

15.2 A note on where to practise

Codeforces is where you compete, so its problems are the ones listed here. But for the very first exposure to each algorithm, the CSES Graph section is better designed: one clean problem per technique, no story, no twist. If you are starting from zero, do the CSES list in parallel. It is at https://cses.fi/problemset/ under "Graph Algorithms", and the first ten problems there map almost exactly onto Parts 3 to 11 of this course.

15.3 Level 1: first graph problems

Goal: get DFS, BFS and DSU into your fingers. Nothing here needs a clever idea.

- **Books Exchange (easy version), 1249B1.** ~1100. https://codeforces.com/problemset/problem/1249/B1 Pattern: functional graph, cycle lengths. Think: `i -> p[i]` gives every vertex outdegree 1. What shape must the graph be?
- **Ice Skating, 217A.** ~1200. https://codeforces.com/problemset/problem/217/A Pattern: components. Think: what exactly is an edge here, and what does the answer `c - 1` mean?
- **Party, 115A.** ~1100. https://codeforces.com/problemset/problem/115/A Pattern: depth in a forest. Think: can you compute each chain length by walking up, and is that fast enough?
- **DZY Loves Chemistry, 445B.** ~1300. https://codeforces.com/problemset/problem/445/B Pattern: components with DSU. Think: what is the contribution of a component of size k?

- **Fox And Two Dots, 510B.** ~1500 but conceptually easy. https://codeforces.com/problemset/problem/510/B Pattern: cycle detection on a grid. Think: which neighbour must you skip, and why is that not enough with multi-edges (here, is it a problem)?

15.4 Level 2: easy

Goal: apply one standard algorithm to a lightly disguised setting.

- **Cyclic Components, 977E.** ~1500. https://codeforces.com/problemset/problem/977/E Pattern: components plus a degree test. Think: what is true about the degree of every vertex in a component that is exactly a cycle?
- **Kefa and Park, 580C.** ~1500. https://codeforces.com/problemset/problem/580/C Pattern: DFS on a tree carrying state downward. Think: what does a child need to be told by its parent?
- **Ring Road, 24A.** ~1400. https://codeforces.com/problemset/problem/24/A Pattern: DFS on a cycle, orientation cost. Think: the graph is a single cycle. How many valid orientations are there?
- **Mahmoud and Ehab and the bipartiteness, 862B.** ~1300. https://codeforces.com/problemset/problem/862/B Pattern: two colouring a tree, then counting. Think: a tree is always bipartite. How many edges could a complete bipartite graph on those sides have?
- **Cover it!, 1176E.** ~1500. https://codeforces.com/problemset/problem/1176/E Pattern: BFS layers and parity. Think: why is one of the two parity classes guaranteed to be small enough?
- **Dijkstra?, 20C.** ~1900 label, but it is plain Dijkstra. https://codeforces.com/problemset/problem/20/C Pattern: Dijkstra with path reconstruction. Think: which type do the distances need, and what do you print if there is no route?
- **Roads not only in Berland, 25D.** ~1900 label, easy idea. https://codeforces.com/problemset/problem/25/D Pattern: DSU. Think: an edge that joins two vertices already connected is spare. How many spares do you need?
- **New Year Permutation, 500B.** ~1500. https://codeforces.com/problemset/problem/500/B Pattern: DSU on positions. Think: within one group, which values can end up where?

15.5 Level 3: easy plus

Goal: combine two ideas, or build the graph yourself.

- **The Two Routes, 601A.** ~1500. https://codeforces.com/problemset/problem/601/A Pattern: BFS on a graph and on its complement. Think: one of the two vehicles always has a direct route. Why?
- **Connect, 1130C.** ~1700. https://codeforces.com/problemset/problem/1130/C Pattern: two BFS runs plus a small search. Think: how many tunnels can you possibly need?
- **Fox And Names, 510C.** ~1600. https://codeforces.com/problemset/problem/510/C Pattern: build a DAG from constraints, then topologically sort. Think: which pair of adjacent names gives you an edge, and which case makes the answer "Impossible" immediately?
- **Graph Without Long Directed Paths, 1144F.** ~1700. https://codeforces.com/problemset/problem/1144/F Pattern: two colouring, then orient by colour. Think: if no directed path of length 2 may exist, what must be true about the endpoints of each edge?
- **Solve The Maze, 1365D.** ~1700. https://codeforces.com/problemset/problem/1365/D Pattern:

grid reasoning plus flood fill. Think: what is the greedy thing to block, and when is it already impossible?

- **Police Stations, 796D.** ~1900. https://codeforces.com/problemset/problem/796/D Pattern: multi-source BFS on a tree. Think: which edges can be deleted, in terms of which source claimed each endpoint?
- **Distance in Tree, 161D.** ~1800. https://codeforces.com/problemset/problem/161/D Pattern: subtree DP with a small second dimension. Think: `dp[v][d]` counts what, and how do you combine two children without double counting?
- **Civilization, 455C.** ~2100 label, very instructive. https://codeforces.com/problemset/problem/455/C Pattern: DSU storing the diameter of each component. Think: when two trees are joined optimally, what is the new diameter in terms of the two old ones?

15.6 Level 4: medium

Goal: the recurring contest patterns.

- **Jzzhu and Cities, 449B.** ~2000. https://codeforces.com/problemset/problem/449/B Pattern: Dijkstra plus counting how many shortest paths reach a vertex. Think: when is a train route redundant?
- **Labyrinth, 1063B.** ~2000. https://codeforces.com/problemset/problem/1063/B Pattern: 0-1 BFS on a grid with a movement budget. Think: what quantity should the search minimise so that both budgets are respected?
- **Three States, 590C.** ~2200. https://codeforces.com/problemset/problem/590/C Pattern: 0-1 BFS from each of three regions, then combine. Think: why is the answer not simply the sum of three pairwise distances?
- **Greg and Graph, 295B.** ~1900. https://codeforces.com/problemset/problem/295/B Pattern: Floyd-Warshall backwards. Think: the deletions run forwards, so what happens if you add the vertices in reverse and use them as the `k` loop?
- **Shichikuji and Power Grid, 1245D.** ~1900. https://codeforces.com/problemset/problem/1245/D Pattern: MST with a virtual vertex. Think: what does an edge from vertex 0 represent?
- **MST Unification, 1108F.** ~2100. https://codeforces.com/problemset/problem/1108/F Pattern: Kruskal, processing equal weights in blocks. Think: when is an edge of weight w replaceable by another of the same weight?
- **Checkposts, 427C.** ~1700. https://codeforces.com/problemset/problem/427/C Pattern: SCC, then a per-component minimum and count. Think: why is it enough to place one checkpost per strongly connected component?
- **Reachability from the Capital, 999E.** ~1900. https://codeforces.com/problemset/problem/999/E Pattern: SCC condensation and indegree counting. Think: after condensing, which components must receive a new edge?
- **Bertown roads, 118E.** ~1900. https://codeforces.com/problemset/problem/118/E Pattern: bridges, then orient along the DFS tree. Think: what does a single bridge do to any orientation?
- **We Need More Bosses, 1000E.** ~2100. https://codeforces.com/problemset/problem/1000/E Pattern: bridges, condense into a tree, then tree diameter. Think: what do the components of "remove all bridges" represent?
- **A and B and Lecture Rooms, 519E.** ~2100. https://codeforces.com/problemset/problem/519/E

Pattern: LCA plus subtree sizes, with careful case work. Think: when is the answer 0, and where is the midpoint of the path?

- **Andrew and Taxi, 1100E.** ~2000. https://codeforces.com/problemset/problem/1100/E Pattern: binary search on the answer plus directed cycle detection. Think: for a fixed limit, which edges are you free to orient?
- **Tree with Maximum Cost, 1092F.** ~1900. https://codeforces.com/problemset/problem/1092/F Pattern: rerooting DP. Think: if you know the answer for the root, how do you get the answer for a child in O(1)?
- **Complete The Graph, 715B.** ~2700, for later. https://codeforces.com/problemset/problem/715/B Pattern: Dijkstra run repeatedly while assigning weights. Attempt only after the rest are comfortable.

15.7 Must Solve

Twenty-five problems covering every technique in this course. If you can solve all of them, you know graphs.

#	Problem	Part	Pattern tested
1	CSES Counting Rooms	5	flood fill
2	CSES Building Roads	5	components, DSU
3	CSES Message Route	4	BFS shortest path
4	CSES Round Trip	6	undirected cycle
5	CSES Course Schedule	8	topological sort
6	CSES Shortest Routes I	11	Dijkstra
7	CSES High Score	11	Bellman-Ford, neg cycle
8	CSES Shortest Routes II	11	Floyd-Warshall
9	Ice Skating, 217A	5	components
10	Party, 115A	12	depth in a forest
11	DZY Loves Chemistry, 445B	9	DSU
12	Cyclic Components, 977E	5/6	degrees plus components
13	Kefa and Park, 580C	12	DFS on a tree
14	Ring Road, 24A	3	DFS on a cycle
15	Mahmoud and Ehab bipartiteness, 862B	7	two colouring
16	Cover it!, 1176E	4/7	BFS parity
17	Fox And Names, 510C	8	build a DAG, topo sort
18	Dijkstra?, 20C	11	Dijkstra with path
19	Roads not only in Berland, 25D	9	DSU, spare edges
20	Labyrinth, 1063B	11	0-1 BFS
21	Greg and Graph, 295B	11	Floyd, reversed
22	Shichikuji and Power Grid, 1245D	10	MST, virtual vertex
23	Checkposts, 427C	13	SCC
24	Bertown roads, 118E	13	bridges
25	We Need More Bosses, 1000E	12/13	bridges plus tree diameter

Coverage check: DFS, BFS, multi-source BFS, 0-1 BFS, components, flood fill, cycle detection in both graph types, bipartite, topological sort, DAG DP, DSU, MST, Dijkstra, Bellman-Ford, Floyd-Warshall,

tree rooting, tree diameter, LCA, bridges, SCC, and graph modelling. That is the whole course.

15.8 A six week plan

Week	Study	Solve
1	Parts 1, 2, 3	Level 1, plus Must Solve 1 to 4
2	Parts 4, 5	Must Solve 3, 9, 12, 16, and Level 2 BFS problems
3	Parts 6, 7, 8	Must Solve 4, 5, 15, 17
4	Parts 9, 10	Must Solve 11, 19, 22, plus Level 2 DSU problems
5	Part 11	Must Solve 6, 7, 8, 18, 20, 21
6	Parts 12, 13, 14	Must Solve 10, 13, 23, 24, 25, plus Level 4

Then redo any problem that took over 45 minutes, one week later, from a blank file.

15.9 Contest checklist

Before you submit a graph solution, read this list once. It takes ten seconds and it saves penalties.

```
[ ] undirected edges pushed both ways
[ ] arrays sized n + 1
[ ] outer loop over all vertices for disconnected input
[ ] long long for any sum of weights
[ ] INF chosen so that INF + w does not overflow
[ ] unreachable case handled in the output
[ ] cleared everything between test cases
[ ] fast IO enabled
[ ] recursion depth is safe, or the code is iterative
[ ] complexity checked against n and m
```

Part 16

Revision Sheet

Every template on a few pages

16.1 Facts in one box

```
n vertices, m edges
sum of degrees = 2m                          (undirected)
tree            = connected, acyclic, m = n - 1
forest with c components                     m = n - c
cycle exists (undirected)                    m > n - c
max edges, simple undirected                 n(n-1)/2
max edges, simple directed                   n(n-1)
DAG                                          directed, no cycle, has a topological order
bipartite                                    no odd cycle
condensation of SCCs                         always a DAG

traversal cost                               O(n + m), never O(n)
recursion depth                              up to n, a path graph is the worst case
```

16.2 Setup and input

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 200005;
const long long INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m; cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);          // delete this line if directed
    }
}
```

Weighted: `vector<vector<pair<int,int>>> adj(n+1);` storing `(neighbour, weight)`.

Grid:

```
int dx[4] = {-1,1,0,0}, dy[4] = {0,0,-1,1};
for (int k = 0; k < 4; k++) {
    int nx = x + dx[k], ny = y + dy[k];
    if (nx < 0 || nx >= R || ny < 0 || ny >= C) continue;
    if (g[nx][ny] == '#') continue;
    ...
}
```

16.3 DFS

```
vector<bool> vis(n + 1, false);

void dfs(int u) {
    vis[u] = true;                          // mark on ENTRY
    for (int v : adj[u])
        if (!vis[v]) dfs(v);
}

for (int i = 1; i <= n; i++) if (!vis[i]) dfs(i);      // all components
```

Iterative:

```
stack<int> st; st.push(s);
while (!st.empty()) {
    int u = st.top(); st.pop();
    if (vis[u]) continue;
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) st.push(v);
}
```

Entry and exit times:

```
tin[u] = timer++;  ... ; tout[u] = timer++;
// u is an ancestor of v  <=>  tin[u] <= tin[v] && tout[v] <= tout[u]
```

16.4 BFS

```
vector<int> d(n + 1, -1);
queue<int> q;
d[s] = 0; q.push(s);

while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u])
        if (d[v] == -1) {
            d[v] = d[u] + 1;
            par[v] = u;
            q.push(v);
```

```
        }
}
```

Multi-source: push every source with `d = 0` before the loop.

Path reconstruction:

```
vector<int> path;
for (int cur = t; cur != 0; cur = par[cur]) path.push_back(cur);
reverse(path.begin(), path.end());
```

16.5 Components and flood fill

```
int comps = 0;
for (int i = 1; i <= n; i++)
    if (!vis[i]) { comps++; dfs(i); }

// edges to connect everything: comps - 1
```

Grid flood fill: same, with the `dx/dy` loop, and use BFS on large grids to avoid stack overflow.

16.6 Cycle detection

Undirected:

```
bool dfs(int u, int p) {
    vis[u] = true;
    for (int v : adj[u]) {
        if (!vis[v]) { if (dfs(v, u)) return true; }
        else if (v != p) return true;
    }
    return false;
}
// with multi-edges, skip by edge id instead of by vertex
```

Directed, three colours:

```
// 0 white, 1 grey (on stack), 2 black (finished)
bool dfs(int u) {
    color[u] = 1;
    for (int v : adj[u]) {
        if (color[v] == 1) return true;                 // back edge
        if (color[v] == 0 && dfs(v)) return true;
    }
    color[u] = 2;
    return false;
}
```

16.7 Bipartite check

```
vector<int> col(n + 1, -1);
for (int s = 1; s <= n; s++) {
    if (col[s] != -1) continue;
    col[s] = 0;
    queue<int> q; q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (col[v] == -1) { col[v] = col[u] ^ 1; q.push(v); }
            else if (col[v] == col[u]) return false;
        }
    }
}
return true;
```

16.8 Topological sort

Kahn:

```
vector<int> indeg(n + 1, 0);
for (int u = 1; u <= n; u++) for (int v : adj[u]) indeg[v]++;

queue<int> q;                                   // priority_queue for lexicographically smallest
for (int i = 1; i <= n; i++) if (!indeg[i]) q.push(i);

vector<int> order;
while (!q.empty()) {
    int u = q.front(); q.pop();
    order.push_back(u);
    for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
}
if ((int)order.size() < n) { /* cycle */ }
```

DFS version: push on finish, then reverse.

DP on a DAG:

```
for (int u : order)
    for (int v : adj[u])
        dp[v] = max(dp[v], dp[u] + 1);          // or  cnt[v] += cnt[u]
```

16.9 DSU

```
int parent[N], sz[N];

void init(int n) { for (int i = 1; i <= n; i++) { parent[i] = i; sz[i] = 1; } }

int find(int x) {
    if (parent[x] == x) return x;
    return parent[x] = find(parent[x]);
```

```
}

bool unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b);
    parent[b] = a;
    sz[a] += sz[b];
    return true;
}
```

Components: start at `n`, decrement on every `unite` that returns true.

16.10 MST

Kruskal:

```
sort(edges.begin(), edges.end());        // by weight
init(n);
long long total = 0; int used = 0;
for (auto& e : edges)
    if (unite(e.u, e.v)) { total += e.w; if (++used == n - 1) break; }
if (used < n - 1) { /* disconnected */ }
```

Prim with a heap:

```
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
pq.push({0, 1});
while (!pq.empty()) {
    auto [w, u] = pq.top(); pq.pop();
    if (inTree[u]) continue;
    inTree[u] = true; total += w;
    for (auto [v, wt] : adj[u]) if (!inTree[v]) pq.push({wt, v});
}
```

Prim $O(n^2)$ for dense or complete graphs: keep `minE[v]`, pick the smallest unused each round.

16.11 Shortest paths

0-1 BFS:

```
deque<int> dq; d[s] = 0; dq.push_front(s);
while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, w] : adj[u])
        if (d[u] + w < d[v]) {
            d[v] = d[u] + w;
            if (w == 0) dq.push_front(v); else dq.push_back(v);
        }
}
```

Dijkstra:

```
vector<long long> d(n + 1, INF);
priority_queue<pair<long long,int>, vector<pair<long long,int>>,
               greater<pair<long long,int>>> pq;
d[s] = 0; pq.push({0, s});

while (!pq.empty()) {
    auto [du, u] = pq.top(); pq.pop();
    if (du > d[u]) continue;                        // stale entry
    for (auto [v, w] : adj[u])
        if (d[u] + w < d[v]) {
            d[v] = d[u] + w;
            pq.push({d[v], v});
        }
}
```

Bellman-Ford:

```
d[s] = 0;
for (int i = 0; i < n - 1; i++)
    for (auto& e : edges)
        if (d[e.u] < INF && d[e.u] + e.w < d[e.v]) d[e.v] = d[e.u] + e.w;

for (auto& e : edges)                               // one extra round
    if (d[e.u] < INF && d[e.u] + e.w < d[e.v]) { /* negative cycle */ }
```

Floyd-Warshall:

```
for (int k = 1; k <= n; k++)                        // k MUST be outermost
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
// d[i][i] < 0 means i lies on a negative cycle
```

16.12 Trees

Rooting, depth, subtree size:

```
void dfs(int u, int p) {
    par[u] = p; sz[u] = 1;
    for (int v : adj[u]) {
        if (v == p) continue;
        dep[v] = dep[u] + 1;
        dfs(v, u);
        sz[u] += sz[v];
    }
}
```

Diameter, two BFS runs:

```
a = farthest_from(1);
diameter = distance_from(a).max();
```

Diameter, one DFS:

```
int height(int u, int p) {
    int t1 = 0, t2 = 0;
    for (int v : adj[u]) {
        if (v == p) continue;
        int h = height(v, u) + 1;
        if (h > t1) { t2 = t1; t1 = h; } else if (h > t2) t2 = h;
    }
    best = max(best, t1 + t2);      // path turning here
    return t1;                      // value for the parent
}
```

LCA by binary lifting:

```
const int LOG = 18;
int up[N][LOG];

void dfs(int u, int p) {
    up[u][0] = p;
    for (int k = 1; k < LOG; k++) up[u][k] = up[up[u][k-1]][k-1];
    for (int v : adj[u]) if (v != p) { dep[v] = dep[u] + 1; dfs(v, u); }
}

int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int diff = dep[u] - dep[v];
    for (int k = 0; k < LOG; k++) if (diff >> k & 1) u = up[u][k];
    if (u == v) return u;
    for (int k = LOG - 1; k >= 0; k--)
        if (up[u][k] != up[v][k]) { u = up[u][k]; v = up[v][k]; }
    return up[u][0];
}

// dist(u,v) = dep[u] + dep[v] - 2 * dep[lca(u,v)]
```

16.13 Bridges, articulation points, SCC

```
void dfs(int u, int p) {                // bridges and APs
    vis[u] = true;
    tin[u] = low[u] = timer++;
    int children = 0;
    for (int v : adj[u]) {
        if (v == p) continue;
        if (vis[v]) {
            low[u] = min(low[u], tin[v]);           // back edge: tin
        } else {
            dfs(v, u);
```

```
            low[u] = min(low[u], low[v]);            // tree edge: low
            if (low[v] >  tin[u]) /* u-v is a bridge */;
            if (low[v] >= tin[u] && p != 0) /* u is an articulation point */;
            children++;
        }
    }
    if (p == 0 && children > 1) /* root is an articulation point */;
}
```

Kosaraju:

```
void dfs1(int u) { vis[u] = true; for (int v : adj[u]) if (!vis[v]) dfs1(v); order.push_back(u); }
void dfs2(int u, int c) { comp[u] = c; for (int v : radj[u]) if (comp[v] == -1) dfs2(v, c); }

for (int i = 1; i <= n; i++) if (!vis[i]) dfs1(i);
int c = 0;
for (int i = n - 1; i >= 0; i--) if (comp[order[i]] == -1) dfs2(order[i], c++);
```

Condensation is a DAG. To make everything strongly connected, the answer is `max(indegree-0 count, outdegree-0 count)`, or 0 if there is one component.

16.14 Complexity cheat sheet

Algorithm	Time	Space
DFS, BFS	O(n + m)	O(n)
Components, flood fill	O(n + m)	O(n)
Cycle detection	O(n + m)	O(n)
Bipartite check	O(n + m)	O(n)
Topological sort	O(n + m)	O(n)
DP on a DAG	O(n + m)	O(n)
DSU operation	almost O(1)	O(n)
Kruskal	O(m log m)	O(n + m)
Prim, heap	O(m log n)	O(n + m)
Prim, dense	$O(n^2)$	$O(n^2)$
0-1 BFS	O(n + m)	O(n)
Dijkstra	O(m log n)	O(n + m)
Bellman-Ford	O(n m)	O(n + m)
Floyd-Warshall	$O(n^3)$	$O(n^2)$
Tree diameter	O(n)	O(n)
LCA binary lifting	O(n log n) build, O(log n) query	O(n log n)
Bridges, articulation points	O(n + m)	O(n)
Kosaraju SCC	O(n + m)	O(n + m)

16.15 Which approach should I think of first

`1. Name the vertices. Name the edges. Write both down.`

```
2. minimum number of equal-cost moves        -> BFS
   distance to the nearest of many sources   -> multi-source BFS
   weights 0 and 1                           -> deque BFS
   non-negative weights                      -> Dijkstra
   negative weights                          -> Bellman-Ford
   all pairs and n <= 500                    -> Floyd
   the graph is a DAG                        -> topological order DP

3. "must come before"                        -> topological sort
   "same group", transitive, online          -> DSU
   "connect everything cheaply"              -> MST
   "which single road or city is critical"   -> bridges, articulation points
   "everyone reaches everyone", directed     -> SCC, then the condensation
   tree, path or subtree queries             -> root it, subtree DP, LCA

4. Does the answer need more than the current vertex?
   Put the extra information into the state, then rerun step 2.

5. Count the states. Compare with the limits. Then code.
```

16.16 The mistakes that cost the most

```
undirected edge pushed only once
missing visited array, or marking it too late
DFS started only from vertex 1
arrays sized n instead of n + 1
not clearing state between test cases
BFS used on a weighted graph
Dijkstra used with negative weights
missing stale check in Dijkstra
int instead of long long for distance sums
INF so large that INF + w overflows
k not the outer loop in Floyd
low[v] used instead of tin[v] on a back edge
> and >= swapped for bridges and articulation points
root case forgotten in articulation points
parent skipped by vertex when multi-edges exist
unreachable case not printed
recursion depth on a path graph of 2 * 10^5 vertices
adjacency matrix on a sparse graph with large n
```

16.17 The four questions to ask on any new graph problem

1. What are the vertices, and what are the edges?
2. Is it directed, is it weighted, can it have cycles?

3. Am I being asked about reachability, distance, ordering, connectivity, or structure?
4. Does the state need to be bigger than “which vertex am I at”?

If you can answer those four on a problem you have never seen, you have learned graphs.